ELEC 374 Digital Systems Engineering

CPU Design Project: Lab Final Report

Author names and student numbers: Russell Litwin (20398808), Jake Hoffman (20398257), Josh Ossip (20398317)

Lab section: Group 26

Submission date: April 1, 2026

Course term: Winter 2026

Institution: Queen's University, Department of Electrical and Computer Engineering

Abstract

This report documents the complete design, implementation, and verification of a Mini SRC processor for the ELEC 374 CPU design project. The processor is a 32 bit single bus architecture with sixteen general purpose registers, a 512 word memory, and a control unit implemented as a finite state machine. The design was developed in four phases, progressing from a standalone datapath through control unit integration to FPGA implementation on a Cyclone V device. The final design executes all required instructions correctly, meets timing at over 250 MHz, and uses 13 percent of the available logic resources. All four phases were completed successfully and demonstrated in the lab.

Table of Contents

1. Project Specification

The Mini SRC is a 32 bit RISC processor with a single internal bus architecture. The design follows the specification provided in the course documentation, which defines the processor state, instruction set, and memory organization. The processor contains sixteen 32 bit general purpose registers R0 through R15, with R0 through R7 designated as general purpose, R8 through R11 as argument registers, R12 as the return address register, R13 and R14 as return value registers, and R15 as the stack pointer. Two additional 32 bit registers HI and LO hold the upper and lower halves of multiplication and division results.

The memory unit is 512 words of 32 bits each. Memory is accessed through the MAR and MDR registers using load and store instructions. A 32 bit input port and a 32 bit output port provide basic input and output capability.

The instruction set consists of 32 bit instructions in five formats. R format instructions encode three register operands for arithmetic and logical operations. I format instructions encode two registers and a 19 bit sign extended constant for immediate operations, loads, and stores. B format instructions encode a register, a two bit condition code, and a 19 bit offset for conditional branches. J format instructions encode a single register for jumps and input output operations. M format instructions encode only an opcode for nop and halt.

The arithmetic logic unit supports thirteen operations: add, subtract, multiply, divide, shift right, shift right arithmetic, shift left, rotate right, rotate left, logical AND, logical OR, negate, and bitwise NOT. The multiply and divide operations produce 64 bit results split across the HI and LO registers.

Four conditional branch modes are supported: branch if zero, branch if nonzero, branch if positive, and branch if negative. The branch target is computed as PC plus one plus the sign extended offset. The processor does not use a condition code register. Instead, the value in the register specified by the Ra field is tested directly against the branch condition.

2. Project Design and Implementation

2.1 Datapath Architecture (Phase 1)

The datapath was implemented as a one bus architecture. In each clock cycle, at most one source value is placed on the shared 32 bit bus, and one or more destination registers

capture that value when their load enable is asserted. A 25 to 1 bus multiplexer selects the bus source based on a five bit control field. The selectable sources include the sixteen general purpose registers, the program counter, instruction register, Y register, MAR, MDR, HI, LO, and the low and high halves of the Z register.

A temporary register Y holds the first arithmetic operand because the bus can carry only one value at a time. The ALU receives Y as its A input and the current bus value as its B input and produces a 64 bit result. A 64 bit Z register captures the ALU output so that both 32 bit operations and 64 bit multiply and divide results can be stored using the same structure. For 32 bit operations the upper half of Z is zero and the result is read from the low half. For multiply and divide the low half goes to the LO register and the high half goes to the HI register.

The ALU was built using dedicated arithmetic blocks. Addition and subtraction share a 32 bit carry lookahead adder constructed from eight four bit CLA blocks. Subtraction is handled by inverting the B operand and setting the carry input to one. Multiplication uses a Booth bit pair recoding multiplier that processes two bits of the multiplier per iteration. Division uses a non restoring division algorithm that produces a 32 bit quotient and 32 bit remainder. Shifts, rotates, and logical operations are implemented using standard Verilog operators.

2.2 Select and Encode, Branches, Memory, and IO (Phase 2)

In Phase 2 the datapath was extended with several components required for full instruction execution. The select and encode logic decodes the register fields Ra, Rb, and Rc from the instruction register and generates one hot register select signals. Three control inputs Gra, Grb, and Grc choose which field to decode, and two outputs Rin decoded and Rout decoded drive the register file load enables and bus source selection respectively. This allows the control unit to reference registers by their instruction field position rather than by explicit register number.

The sign extension unit takes the 19 bit constant field from the instruction register and sign extends it to 32 bits. The sign extended value is placed on the bus when the Cout control signal is asserted, allowing immediate values to participate in ALU operations.

The CON FF logic evaluates branch conditions. It receives the bus value and the two bit condition code from the instruction register and produces a one bit condition flag. The four conditions are branch if zero, branch if nonzero, branch if positive, and branch if negative. The flag is latched on the clock edge when CONin is asserted, and it controls whether the conditional program counter write is enabled during the branch execute state.

The memory subsystem uses a 512 by 32 bit synchronous RAM block. The RAM is initialized from a hex file at elaboration time. Reads are asynchronous and writes occur on the rising clock edge when the write enable signal is asserted. The MAR provides the address and the MDR provides the write data or captures the read data. A separate debug

address port allows the testbench to inspect any memory location without interfering with the program.

The input and output ports are simple registered interfaces. The output port captures the bus value when OutPortin is asserted. The input port captures an external data value when InPortLoad is asserted, and the latched value is placed on the bus when InPortout is asserted.

2.3 Control Unit (Phase 3)

The control unit was implemented as a single Mealy style finite state machine with 43 states. On each clock edge the state register advances according to the next state logic, and a combinational block drives all datapath control signals based on the current state and the opcode field of the instruction register.

Every instruction begins with a three state fetch sequence. In the first fetch state the program counter is placed on the bus and loaded into MAR while the Z register captures PC plus one through the IncPC path. In the second fetch state the incremented value is written back to PC and the MDR captures the instruction word from RAM. In the third fetch state the MDR contents are transferred into the instruction register. A decode state then examines the five bit opcode and branches to the appropriate execution state sequence.

R type ALU instructions use three execution states. The first places the Rb register on the bus and loads it into Y. The second places the Rc register on the bus, sets the ALU operation, and captures the result in Z. The third places the Z low half on the bus and writes it to the Ra register. Immediate ALU instructions follow the same pattern but use the sign extended constant in place of the Rc register.

Multiply and divide use four execution states because the 64 bit result must be split into separate writes to HI and LO. Load instructions use five states to compute the effective address, read from memory, and write the result to a register. Store instructions also use five states but latch the store data into MDR before computing the address.

Branch instructions use four states. The first latches the condition flag by placing the Ra register on the bus and asserting CONin. The remaining states compute the branch target as PC plus the sign extended offset and conditionally write it to PC based on the latched flag. Jump register and jump and link use one and two states respectively. The halt state deasserts the run signal and holds the machine in a self loop.

2.4 FPGA Implementation (Phase 4)

For the FPGA implementation a top level wrapper module was created to interface the CPU with the DE0 CV development board. The 50 MHz board clock is divided down using a three bit counter, with bit two selected as the CPU clock, producing a 6.25 MHz effective clock rate. Push buttons KEY0 and KEY1 are active low on the DE0 CV, so

they are inverted to produce the active high reset and stop signals expected by the control unit.

The eight slide switches SW0 through SW7 feed into the lower eight bits of the 32 bit input port, with the upper 24 bits tied to zero. The lower byte of the output port drives two seven segment display decoders for HEX0 and HEX1, allowing the output value to be displayed in hexadecimal on the board. Displays HEX2 through HEX5 are blanked by driving all segments high. A single LED indicates the run state of the processor.

The test program for Phase 4 extends the Phase 3 program with an input output loop. The program reads the switch value through the input port, stores it to memory, and then enters a display loop that shifts the value right by one bit and outputs each intermediate result. A delay counter loaded from memory provides a visible pause between display updates. The loop repeats five times before outputting a final value and halting. On the FPGA board, the display shows a shifting pattern across the two seven segment digits before settling on the final value.

The Quartus project targets the Cyclone V 5CEBA4F23C7 device. Pin assignments follow the DE0 CV user manual. An SDC constraint file defines the 50 MHz clock on the CLOCK 50 input pin. The design compiles with zero errors and meets timing with positive slack on all paths.

3. Evaluation Results

3.1 Maximum Frequency of Operation

The Quartus Timing Analyzer reports a worst case setup slack of 16.024 nanoseconds against a 20 nanosecond clock period under the slow 1100 millivolt 85 degree Celsius model. The maximum operating frequency is therefore one divided by the critical path delay of 3.976 nanoseconds, which gives approximately 251.5 MHz. Under the fast corner model the slack increases to 17.927 nanoseconds, corresponding to a maximum frequency of approximately 482.6 MHz. In practice the design runs on the FPGA at a divided clock rate of 6.25 MHz, well within the timing margins.

Corner	Setup Slack (ns)	Fmax (MHz)
Slow 1100mV 85C	16.024	251.5
Fast 1100mV 0C	17.927	482.6

3.2 Average Cycles Per Instruction

The Phase 3 test program executes 44 instructions in 310 clock cycles, giving an average CPI of 7.05. This includes the four cycle fetch and decode overhead for every instruction plus the variable number of execution states per instruction type. Instructions with memory access such as load and store have the highest individual cycle counts at nine

cycles each, while simple register moves through mfhi and mflo complete in five cycles. The overall average is dominated by the fixed four cycle fetch decode cost, which accounts for more than half of the total cycles.

The Phase 4 test program executes approximately 782 instructions in 5269 clock cycles, giving an average CPI of approximately 6.74. The lower average compared to Phase 3 is due to the inner delay loop, which consists of short instructions that execute in fewer cycles per instruction.

Test Program	Instructions	Cycles	Avg CPI
Phase 3	44	310	7.05
Phase 4	~782	5269	~6.74

3.3 Chip Area Utilization

The design uses 2344 of 18480 available adaptive logic modules on the Cyclone V device, which corresponds to 13 percent utilization. The register count is 937. The RAM block usage is two blocks out of 308 available, consuming 16384 of over three million available block memory bits. No DSP blocks or PLLs are used. The low utilization indicates substantial headroom for additional features such as a wider bus architecture or more complex control logic.

Resource	Used / Available	Utilization
Logic (ALMs)	2,344 / 18,480	13%
Registers	937	-
Block Memory Bits	16,384 / 3,153,920	<1%
RAM Blocks	2 / 308	<1%

4. Discussion

The single bus architecture was straightforward to implement and debug because each clock cycle involves exactly one bus transfer. The main disadvantage is performance. Every instruction requires at least four cycles for fetch and decode before any execution states begin. A multi bus architecture could overlap operand reads and reduce the cycle count for most instructions, but at the cost of a more complex bus arbitration scheme and wider multiplexers.

The non restoring division algorithm was the most challenging component to implement. Early versions produced incorrect quotients for certain sign combinations because the correction step was not handling the remainder sign properly. The issue was resolved by testing all four sign combinations in the Phase 1 testbench and comparing against hand calculated expected values.

The Booth bit pair multiplier was chosen over a simple shift and add multiplier because it reduces the number of partial product rows by half. This did not significantly affect the synthesis area because the multiplier is purely combinational and Quartus optimizes it aggressively, but it simplified the verification since each partial product step could be checked against the recoding table.

One challenge during Phase 3 was ensuring that the control unit and datapath agreed on timing. The control unit drives its signals combinationally based on the current state, and the datapath latches values on the next clock edge. Off by one errors in the state sequencing caused several instructions to read stale register values during initial testing. These were caught by the automated testbench, which checks not just final values but also intermediate control flow properties like whether specific instructions were observed in the IR during execution.

The Phase 4 FPGA implementation required a clock divider because the test program runs too quickly at 50 MHz for the display pattern to be visible. A three bit counter with bit two as the CPU clock was sufficient to slow the display updates to a rate where the shifting pattern could be observed by eye.

All four phases were completed and demonstrated successfully with the exception of a branching problem in phase 2. Since phase 2 this problem has been fixed and there are no known outstanding issues. The design passes all testbench verification checks for both Phase 3 and Phase 4 test programs.

5. Conclusion and Future Work

The Mini SRC processor was designed, simulated, and implemented on the DE0 CV FPGA board across four project phases. The final design correctly executes all required instructions including arithmetic and logical operations, load and store with multiple addressing modes, conditional branches, jump and link, multiply and divide with HI LO register writeback, and input output operations. The control unit finite state machine generates the correct control sequence for each instruction, and the automated testbenches confirm correct behavior for both the Phase 3 and Phase 4 test programs.

The design meets timing with a maximum operating frequency of approximately 251 MHz under worst case conditions and uses 13 percent of the available logic on the Cyclone V device. The average cycles per instruction is approximately seven, which is expected for a single bus architecture where every instruction requires a four cycle fetch and decode sequence.

Several directions for future work could improve the design. A three bus architecture would allow simultaneous reads of two source operands and a write to the destination in a single cycle, potentially reducing the CPI by half or more. Pipelining the fetch decode execute stages would allow the next instruction to begin fetching while the current one is

still executing. Branch prediction could reduce the penalty of taken branches in a pipelined design. Hardware interrupt support would allow the processor to respond to external events without polling. A cache memory system would be necessary if the address space were expanded beyond the current 512 words.

Academic Integrity Statement

"We do hereby verify that this written lab report is our own work and contains our own original ideas, concepts, and designs. No portion of this report has been copied in whole or in part from another source, with the possible exception of properly referenced material".

Appendix A: Verilog Source Code

The following listings contain the complete Verilog source code for the Mini SRC processor design.

A.1 Defines (defines.vh)

```
`ifndef DEFINES_VH
`define DEFINES_VH

`define ANDop   4'd1
`define ORop    4'd2
`define SHLop   4'd3
`define SHRop   4'd4
`define SHRAop  4'd5
`define ROLop   4'd6
`define RORop   4'd7
`define ADDop   4'd8
`define SUBop   4'd9
`define MULop   4'd10
`define DIVop   4'd11
`define NEGop   4'd12
`define NOTop   4'd13

`define OP_ADD   5'd0
`define OP_SUB   5'd1
`define OP_AND   5'd2
`define OP_OR    5'd3
`define OP_SHR   5'd4
`define OP_SHRA  5'd5
`define OP_SHL   5'd6
`define OP_ROR   5'd7
`define OP_ROL   5'd8
`define OP_ADDI  5'd9
`define OP_ANDI  5'd10
`define OP_ORI   5'd11
`define OP_DIV   5'd12
`define OP_MUL   5'd13
`define OP_NEG   5'd14
`define OP_NOT   5'd15
`define OP_LD    5'd16
`define OP_LDI   5'd17
`define OP_ST    5'd18
`define OP_JAL   5'd19
`define OP_JR    5'd20
`define OP_BR    5'd21
`define OP_IN    5'd22
`define OP_OUT   5'd23
`define OP_MFHI  5'd24
```

```
`define OP_MFLO  5'd25
`define OP_NOP   5'd26
`define OP_HALT  5'd27

`endif
```

A.2 Control Unit (control_unit.v)

```
`include "defines.vh"

module control_unit (
    input  wire        clk,
    input  wire        reset,
    input  wire        stop,
    input  wire [31:0] IR,
    input  wire        CON_q,

    output wire        Run,

    output reg  [4:0]  bus_sel,
    output reg  [15:0] Rin,
    output reg         Yin,
    output reg         Zin,
    output reg         PCin,
    output reg         IRin,
    output reg         MARin,
    output reg         MDRin,
    output reg         HIin,
    output reg         LOin,
    output reg         IncPC,
    output reg         Read,
    output reg         Write,

    output reg         Gra,
    output reg         Grb,
    output reg         Grc,
    output reg         Rin_dec,
    output reg         Rout_dec,
    output reg         BAout,
    output reg         Cout,
    output reg         CONin,
    output reg         CON_to_PCin,
    output reg         OutPortin,
    output reg         InPortout,
    output reg         InPortLoad,

    output reg  [3:0]  op
);

    localparam [4:0] SEL_PC    = 5'd16;
    localparam [4:0] SEL_MDR   = 5'd20;
    localparam [4:0] SEL_HI    = 5'd21;
    localparam [4:0] SEL_LO    = 5'd22;
    localparam [4:0] SEL_ZLOW  = 5'd23;
    localparam [4:0] SEL_ZHIGH = 5'd24;

    localparam [5:0]
        S_RESET      = 6'd0,
        S_FETCH0     = 6'd1,
        S_FETCH1     = 6'd2,
        S_FETCH2     = 6'd3,
        S_DECODE     = 6'd42,
        S_ALU_T3     = 6'd4,
        S_ALU_T4     = 6'd5,
        S_ALU_T5     = 6'd6,
        S_UNARY_T3   = 6'd7,
        S_UNARY_T4   = 6'd8,
        S_MD_T3      = 6'd9,
        S_MD_T4      = 6'd10,
        S_MD_T5      = 6'd11,
        S_MD_T6      = 6'd12,
        S_LD_T3      = 6'd13,
```

```
    S_LD_T4      = 6'd14,
    S_LD_T5      = 6'd15,
    S_LD_T6      = 6'd16,
    S_LD_T7      = 6'd17,
    S_LDI_T3     = 6'd18,
    S_LDI_T4     = 6'd19,
    S_LDI_T5     = 6'd20,
    S_ST_T3      = 6'd21,
    S_ST_T4      = 6'd22,
    S_ST_T5      = 6'd23,
    S_ST_T6      = 6'd24,
    S_ST_T7      = 6'd25,
    S_IMM_T3     = 6'd26,
    S_IMM_T4     = 6'd27,
    S_IMM_T5     = 6'd28,
    S_BR_T3      = 6'd29,
    S_BR_T4      = 6'd30,
    S_BR_T5      = 6'd31,
    S_BR_T6      = 6'd32,
    S_JR_T3      = 6'd33,
    S_JAL_T3     = 6'd34,
    S_JAL_T4     = 6'd35,
    S_MFHILO_T3  = 6'd36,
    S_OUT_T3     = 6'd37,
    S_IN_T3      = 6'd38,
    S_IN_T4      = 6'd39,
    S_NOP_T3     = 6'd40,
    S_HALT       = 6'd41;

reg [5:0] state;
reg [5:0] next_state;
reg       run_q;
reg       next_run;

wire [4:0] opcode = IR[31:27];

assign Run = run_q;

function [3:0] alu_from_opcode;
    input [4:0] opcode_f;
    begin
        case (opcode_f)
            `OP_ADD:  alu_from_opcode = `ADDop;
            `OP_SUB:  alu_from_opcode = `SUBop;
            `OP_AND:  alu_from_opcode = `ANDop;
            `OP_OR:   alu_from_opcode = `ORop;
            `OP_SHR:  alu_from_opcode = `SHRop;
            `OP_SHRA: alu_from_opcode = `SHRAop;
            `OP_SHL:  alu_from_opcode = `SHLop;
            `OP_ROR:  alu_from_opcode = `RORop;
            `OP_ROL:  alu_from_opcode = `ROLop;
            `OP_ADDI: alu_from_opcode = `ADDop;
            `OP_ANDI: alu_from_opcode = `ANDop;
            `OP_ORI:  alu_from_opcode = `ORop;
            `OP_MUL:  alu_from_opcode = `MULop;
            `OP_DIV:  alu_from_opcode = `DIVop;
            `OP_NEG:  alu_from_opcode = `NEGop;
            `OP_NOT:  alu_from_opcode = `NOTop;
            default:  alu_from_opcode = 4'd0;
        endcase
    end
endfunction

always @(posedge clk) begin
    if (reset) begin
        state  <= S_RESET;
        run_q  <= 1'b1;
    end else begin
        state  <= next_state;
        run_q  <= next_run;
    end
end

always @(*) begin
```

```
next_state = state;
next_run   = run_q;

bus_sel     = 5'd0;
Rin         = 16'd0;
Yin         = 1'b0;
Zin         = 1'b0;
PCin        = 1'b0;
IRin        = 1'b0;
MARin       = 1'b0;
MDRin       = 1'b0;
HIin        = 1'b0;
LOin        = 1'b0;
IncPC       = 1'b0;
Read        = 1'b0;
Write       = 1'b0;
Gra         = 1'b0;
Grb         = 1'b0;
Grc         = 1'b0;
Rin_dec     = 1'b0;
Rout_dec    = 1'b0;
BAout       = 1'b0;
Cout        = 1'b0;
CONin       = 1'b0;
CON_to_PCin = 1'b0;
OutPortin   = 1'b0;
InPortout   = 1'b0;
InPortLoad  = 1'b0;
op          = 4'd0;

if (stop) begin
    next_state = S_HALT;
    next_run   = 1'b0;
end else begin
    case (state)
        S_RESET: begin
            next_state = S_FETCH0;
            next_run   = 1'b1;
        end

        S_FETCH0: begin
            bus_sel   = SEL_PC;
            MARin     = 1'b1;
            IncPC     = 1'b1;
            Zin       = 1'b1;
            next_state = S_FETCH1;
        end

        S_FETCH1: begin
            bus_sel   = SEL_ZLOW;
            PCin      = 1'b1;
            Read      = 1'b1;
            MDRin     = 1'b1;
            next_state = S_FETCH2;
        end

        S_FETCH2: begin
            bus_sel   = SEL_MDR;
            IRin      = 1'b1;
            next_state = S_DECODE;
        end

        S_DECODE: begin
            case (opcode)
                `OP_ADD,
                `OP_SUB,
                `OP_AND,
                `OP_OR,
                `OP_SHR,
                `OP_SHRA,
                `OP_SHL,
                `OP_ROR,
                `OP_ROL:  next_state = S_ALU_T3;
```

```
        `OP_NEG,
        `OP_NOT:  next_state = S_UNARY_T3;

        `OP_MUL,
        `OP_DIV:  next_state = S_MD_T3;

        `OP_LD:   next_state = S_LD_T3;
        `OP_LDI:  next_state = S_LDI_T3;
        `OP_ST:   next_state = S_ST_T3;

        `OP_ADDI,
        `OP_ANDI,
        `OP_ORI:  next_state = S_IMM_T3;

        `OP_BR:   next_state = S_BR_T3;
        `OP_JR:   next_state = S_JR_T3;
        `OP_JAL:  next_state = S_JAL_T3;

        `OP_MFHI,
        `OP_MFLO: next_state = S_MFHILO_T3;

        `OP_OUT:  next_state = S_OUT_T3;
        `OP_IN:   next_state = S_IN_T3;

        `OP_NOP:  next_state = S_NOP_T3;

        `OP_HALT: begin
            next_state = S_HALT;
            next_run   = 1'b0;
        end

        default: begin
            next_state = S_HALT;
            next_run   = 1'b0;
        end
    endcase
end

S_ALU_T3: begin
    Grb       = 1'b1;
    Rout_dec  = 1'b1;
    Yin       = 1'b1;
    next_state = S_ALU_T4;
end

S_ALU_T4: begin
    Grc       = 1'b1;
    Rout_dec  = 1'b1;
    op        = alu_from_opcode(opcode);
    Zin       = 1'b1;
    next_state = S_ALU_T5;
end

S_ALU_T5: begin
    bus_sel   = SEL_ZLOW;
    Gra       = 1'b1;
    Rin_dec   = 1'b1;
    next_state = S_FETCH0;
end

S_UNARY_T3: begin
    Grb       = 1'b1;
    Rout_dec  = 1'b1;
    op        = alu_from_opcode(opcode);
    Zin       = 1'b1;
    next_state = S_UNARY_T4;
end

S_UNARY_T4: begin
    bus_sel   = SEL_ZLOW;
    Gra       = 1'b1;
    Rin_dec   = 1'b1;
    next_state = S_FETCH0;
end
```

```
S_MD_T3: begin
    Gra       = 1'b1;
    Rout_dec  = 1'b1;
    Yin       = 1'b1;
    next_state = S_MD_T4;
end

S_MD_T4: begin
    Grb       = 1'b1;
    Rout_dec  = 1'b1;
    op        = alu_from_opcode(opcode);
    Zin       = 1'b1;
    next_state = S_MD_T5;
end

S_MD_T5: begin
    bus_sel   = SEL_ZLOW;
    LOin      = 1'b1;
    next_state = S_MD_T6;
end

S_MD_T6: begin
    bus_sel   = SEL_ZHIGH;
    HIin      = 1'b1;
    next_state = S_FETCH0;
end

S_LD_T3: begin
    Grb       = 1'b1;
    Rout_dec  = 1'b1;
    BAout     = 1'b1;
    Yin       = 1'b1;
    next_state = S_LD_T4;
end

S_LD_T4: begin
    Cout      = 1'b1;
    op        = `ADDop;
    Zin       = 1'b1;
    next_state = S_LD_T5;
end

S_LD_T5: begin
    bus_sel   = SEL_ZLOW;
    MARin     = 1'b1;
    next_state = S_LD_T6;
end

S_LD_T6: begin
    Read      = 1'b1;
    MDRin     = 1'b1;
    next_state = S_LD_T7;
end

S_LD_T7: begin
    bus_sel   = SEL_MDR;
    Gra       = 1'b1;
    Rin_dec   = 1'b1;
    next_state = S_FETCH0;
end

S_LDI_T3: begin
    Grb       = 1'b1;
    Rout_dec  = 1'b1;
    BAout     = 1'b1;
    Yin       = 1'b1;
    next_state = S_LDI_T4;
end

S_LDI_T4: begin
    Cout      = 1'b1;
    op        = `ADDop;
    Zin       = 1'b1;
```

```
    next_state = S_LDI_T5;
end

S_LDI_T5: begin
    bus_sel   = SEL_ZLOW;
    Gra       = 1'b1;
    Rin_dec   = 1'b1;
    next_state = S_FETCH0;
end

S_ST_T3: begin
    Gra       = 1'b1;
    Rout_dec  = 1'b1;
    MDRin     = 1'b1;
    next_state = S_ST_T4;
end

S_ST_T4: begin
    Grb       = 1'b1;
    Rout_dec  = 1'b1;
    BAout     = 1'b1;
    Yin       = 1'b1;
    next_state = S_ST_T5;
end

S_ST_T5: begin
    Cout      = 1'b1;
    op        = `ADDop;
    Zin       = 1'b1;
    next_state = S_ST_T6;
end

S_ST_T6: begin
    bus_sel   = SEL_ZLOW;
    MARin     = 1'b1;
    next_state = S_ST_T7;
end

S_ST_T7: begin
    Write     = 1'b1;
    next_state = S_FETCH0;
end

S_IMM_T3: begin
    Grb       = 1'b1;
    Rout_dec  = 1'b1;
    Yin       = 1'b1;
    next_state = S_IMM_T4;
end

S_IMM_T4: begin
    Cout      = 1'b1;
    op        = alu_from_opcode(opcode);
    Zin       = 1'b1;
    next_state = S_IMM_T5;
end

S_IMM_T5: begin
    bus_sel   = SEL_ZLOW;
    Gra       = 1'b1;
    Rin_dec   = 1'b1;
    next_state = S_FETCH0;
end

S_BR_T3: begin
    Gra       = 1'b1;
    Rout_dec  = 1'b1;
    CONin     = 1'b1;
    next_state = S_BR_T4;
end

S_BR_T4: begin
    bus_sel   = SEL_PC;
    Yin       = 1'b1;
```

```
    next_state = S_BR_T5;
end

S_BR_T5: begin
    Cout      = 1'b1;
    op        = `ADDop;
    Zin       = 1'b1;
    next_state = S_BR_T6;
end

S_BR_T6: begin
    bus_sel     = SEL_ZLOW;
    CON_to_PCin = 1'b1;
    next_state  = S_FETCH0;
end

S_JR_T3: begin
    Gra       = 1'b1;
    Rout_dec  = 1'b1;
    PCin      = 1'b1;
    next_state = S_FETCH0;
end

S_JAL_T3: begin
    bus_sel    = SEL_PC;
    Rin[12]    = 1'b1;
    next_state = S_JAL_T4;
end

S_JAL_T4: begin
    Gra       = 1'b1;
    Rout_dec  = 1'b1;
    PCin      = 1'b1;
    next_state = S_FETCH0;
end

S_MFHILO_T3: begin
    bus_sel   = (opcode == `OP_MFHI) ? SEL_HI : SEL_LO;
    Gra       = 1'b1;
    Rin_dec   = 1'b1;
    next_state = S_FETCH0;
end

S_OUT_T3: begin
    Gra       = 1'b1;
    Rout_dec  = 1'b1;
    OutPortin = 1'b1;
    next_state = S_FETCH0;
end

S_IN_T3: begin
    InPortLoad = 1'b1;
    next_state = S_IN_T4;
end

S_IN_T4: begin
    InPortout  = 1'b1;
    Gra        = 1'b1;
    Rin_dec    = 1'b1;
    next_state = S_FETCH0;
end

S_NOP_T3: begin
    next_state = S_FETCH0;
end

S_HALT: begin
    next_state = S_HALT;
    next_run   = 1'b0;
end

default: begin
    next_state = S_HALT;
    next_run   = 1'b0;
```

```
                end
            endcase
        end
    end

endmodule
```

A.3 CPU Integration (cpu_phase3.v)

```
module cpu_phase3 #(
    parameter MEM_INIT_FILE = ""
) (
    input  wire        clk,
    input  wire        reset,
    input  wire        stop,
    input  wire [31:0] in_port_data,
    input  wire [8:0]  mem_dbg_addr,

    output wire        run,
    output wire [31:0] bus_dbg,
    output wire [31:0] ir_dbg,
    output wire [31:0] pc_dbg,
    output wire [31:0] mar_dbg,
    output wire [31:0] mdr_dbg,
    output wire [31:0] hi_dbg,
    output wire [31:0] lo_dbg,
    output wire [31:0] y_dbg,
    output wire [63:0] z_dbg,
    output wire        con_dbg,
    output wire [31:0] out_port_dbg,
    output wire [31:0] in_port_dbg,
    output wire [31:0] ram_read_data_dbg,
    output wire [31:0] ram_dbg_data,
    output wire [31:0] r0_dbg,
    output wire [31:0] r1_dbg,
    output wire [31:0] r2_dbg,
    output wire [31:0] r3_dbg,
    output wire [31:0] r4_dbg,
    output wire [31:0] r5_dbg,
    output wire [31:0] r6_dbg,
    output wire [31:0] r7_dbg,
    output wire [31:0] r8_dbg,
    output wire [31:0] r9_dbg,
    output wire [31:0] r10_dbg,
    output wire [31:0] r11_dbg,
    output wire [31:0] r12_dbg,
    output wire [31:0] r13_dbg,
    output wire [31:0] r14_dbg,
    output wire [31:0] r15_dbg
);

    wire [4:0]  bus_sel;
    wire [15:0] Rin;
    wire        Yin;
    wire        Zin;
    wire        PCin;
    wire        IRin;
    wire        MARin;
    wire        MDRin;
    wire        HIin;
    wire        LOin;
    wire        IncPC;
    wire        Read;
    wire        Write;
    wire        Gra;
    wire        Grb;
    wire        Grc;
    wire        Rin_dec;
    wire        Rout_dec;
    wire        BAout;
    wire        Cout;
    wire        CONin;
```

```
wire        CON_to_PCin;
wire        OutPortin;
wire        InPortout;
wire        InPortLoad;
wire [3:0]  op;

control_unit UCTRL (
    .clk(clk),
    .reset(reset),
    .stop(stop),
    .IR(ir_dbg),
    .CON_q(con_dbg),
    .Run(run),
    .bus_sel(bus_sel),
    .Rin(Rin),
    .Yin(Yin),
    .Zin(Zin),
    .PCin(PCin),
    .IRin(IRin),
    .MARin(MARin),
    .MDRin(MDRin),
    .HIin(HIin),
    .LOin(LOin),
    .IncPC(IncPC),
    .Read(Read),
    .Write(Write),
    .Gra(Gra),
    .Grb(Grb),
    .Grc(Grc),
    .Rin_dec(Rin_dec),
    .Rout_dec(Rout_dec),
    .BAout(BAout),
    .Cout(Cout),
    .CONin(CONin),
    .CON_to_PCin(CON_to_PCin),
    .OutPortin(OutPortin),
    .InPortout(InPortout),
    .InPortLoad(InPortLoad),
    .op(op)
);

datapath_logic #(
    .MEM_INIT_FILE(MEM_INIT_FILE)
) UDATAPATH (
    .clk(clk),
    .reset(reset),

    .bus_sel(bus_sel),
    .Rin(Rin),
    .Yin(Yin),
    .Zin(Zin),

    .PCin(PCin),
    .IRin(IRin),
    .MARin(MARin),
    .MDRin(MDRin),
    .HIin(HIin),
    .LOin(LOin),
    .IncPC(IncPC),
    .Read(Read),
    .Write(Write),
    .Mdatain(32'b0),

    .Gra(Gra),
    .Grb(Grb),
    .Grc(Grc),
    .Rin_dec(Rin_dec),
    .Rout_dec(Rout_dec),
    .BAout(BAout),
    .Cout(Cout),
    .CONin(CONin),
    .CON_to_PCin(CON_to_PCin),
    .OutPortin(OutPortin),
    .InPortout(InPortout),
```

```
        .InPortLoad(InPortLoad),
        .InPort_data(in_port_data),
        .UseRAM(1'b1),
        .MemDbgAddr(mem_dbg_addr),

        .PC(32'b0),
        .IR(32'b0),
        .HI(32'b0),
        .LO(32'b0),
        .MAR(32'b0),
        .MDR(32'b0),

        .op(op),

        .BUS(bus_dbg),
        .R0_q(r0_dbg),
        .R1_q(r1_dbg),
        .R2_q(r2_dbg),
        .R3_q(r3_dbg),
        .R4_q(r4_dbg),
        .R5_q(r5_dbg),
        .R6_q(r6_dbg),
        .R7_q(r7_dbg),
        .R8_q(r8_dbg),
        .R9_q(r9_dbg),
        .R10_q(r10_dbg),
        .R11_q(r11_dbg),
        .R12_q(r12_dbg),
        .R13_q(r13_dbg),
        .R14_q(r14_dbg),
        .R15_q(r15_dbg),
        .HI_q_dbg(hi_dbg),
        .LO_q_dbg(lo_dbg),
        .IR_q_dbg(ir_dbg),
        .PC_q_dbg(pc_dbg),
        .MAR_q_dbg(mar_dbg),
        .MDR_q_dbg(mdr_dbg),
        .Y_q(y_dbg),
        .Z_q(z_dbg),
        .CON_q_dbg(con_dbg),
        .OutPort_q_dbg(out_port_dbg),
        .InPort_q_dbg(in_port_dbg),
        .RAM_read_data_dbg(ram_read_data_dbg),
        .RAM_dbg_data(ram_dbg_data)
    );

endmodule
```

A.4 FPGA Top Level (cpu_phase4_top.v)

```
module cpu_phase4_top (
    input  wire        CLOCK_50,
    input  wire [1:0]  KEY,
    input  wire [7:0]  SW,
    output wire        RUN_LED,
    output wire [6:0]  HEX0,
    output wire [6:0]  HEX1,
    output wire [6:0]  HEX2,
    output wire [6:0]  HEX3,
    output wire [6:0]  HEX4,
    output wire [6:0]  HEX5
);
    localparam USE_RAW_CLOCK = 1'b0;
    localparam integer CLOCK_DIVIDER_BIT = 2; // 50 MHz / 2^(2+1) = 6.25 MHz

    reg [31:0] clock_divider = 32'b0;
    wire       cpu_clk;
    wire       reset;
    wire       stop;
    wire       run;
    wire [31:0] out_port_dbg;
```

```
    always @(posedge CLOCK_50) begin
        clock_divider <= clock_divider + 32'd1;
    end

    assign cpu_clk = USE_RAW_CLOCK ? CLOCK_50 : clock_divider[CLOCK_DIVIDER_BIT];
    assign reset   = ~KEY[0];
    assign stop    = ~KEY[1];
    assign RUN_LED = run;

    seven_seg_hex u_hex0 (
        .hex(out_port_dbg[3:0]),
        .seg_n(HEX0)
    );

    seven_seg_hex u_hex1 (
        .hex(out_port_dbg[7:4]),
        .seg_n(HEX1)
    );

    assign HEX2 = 7'b1111111;
    assign HEX3 = 7'b1111111;
    assign HEX4 = 7'b1111111;
    assign HEX5 = 7'b1111111;

    cpu_phase3 #(
        .MEM_INIT_FILE("tb/phase4_mem_init.hex")
    ) dut (
        .clk(cpu_clk),
        .reset(reset),
        .stop(stop),
        .in_port_data({24'b0, SW}),
        .mem_dbg_addr(9'd0),
        .run(run),
        .bus_dbg(),
        .ir_dbg(),
        .pc_dbg(),
        .mar_dbg(),
        .mdr_dbg(),
        .hi_dbg(),
        .lo_dbg(),
        .y_dbg(),
        .z_dbg(),
        .con_dbg(),
        .out_port_dbg(out_port_dbg),
        .in_port_dbg(),
        .ram_read_data_dbg(),
        .ram_dbg_data(),
        .r0_dbg(),
        .r1_dbg(),
        .r2_dbg(),
        .r3_dbg(),
        .r4_dbg(),
        .r5_dbg(),
        .r6_dbg(),
        .r7_dbg(),
        .r8_dbg(),
        .r9_dbg(),
        .r10_dbg(),
        .r11_dbg(),
        .r12_dbg(),
        .r13_dbg(),
        .r14_dbg(),
        .r15_dbg()
    );
endmodule
```

A.5 Datapath (datapath_logic.v)

```
module datapath_logic #(
    parameter MEM_INIT_FILE = ""
) (
    input   wire        clk,
```

```
input   wire        reset,

// bus control
input   wire [4:0]  bus_sel,
input   wire [15:0] Rin,
input   wire        Yin,
input   wire        Zin,

// control of internal special registers
input   wire        PCin,
input   wire        IRin,
input   wire        MARin,
input   wire        MDRin,
input   wire        HIin,
input   wire        LOin,
input   wire        IncPC,
input   wire        Read,
input   wire        Write,
input   wire [31:0] Mdatain,

// phase 2 controls
input   wire        Gra,
input   wire        Grb,
input   wire        Grc,
input   wire        Rin_dec,
input   wire        Rout_dec,
input   wire        BAout,
input   wire        Cout,
input   wire        CONin,
input   wire        CON_to_PCin,
input   wire        OutPortin,
input   wire        InPortout,
input   wire        InPortLoad,
input   wire [31:0] InPort_data,
input   wire        UseRAM,
input   wire [8:0]  MemDbgAddr,

// reset-time seed values for internal special registers
input   wire [31:0] PC,
input   wire [31:0] IR,
input   wire [31:0] HI,
input   wire [31:0] LO,
input   wire [31:0] MAR,
input   wire [31:0] MDR,

// ALU logic controls
input   wire [3:0]  op,

// debug/visibility
output  wire [31:0] BUS,
output  wire [31:0] R5_q,
output  wire [31:0] R6_q,
output  wire [31:0] R3_q,
output  wire [31:0] R2_q,
output  wire [31:0] R0_q,
output  wire [31:0] R1_q,
output  wire [31:0] R4_q,
output  wire [31:0] R12_q,
output  wire [31:0] R7_q,
output  wire [31:0] R8_q,
output  wire [31:0] R9_q,
output  wire [31:0] R10_q,
output  wire [31:0] R11_q,
output  wire [31:0] R13_q,
output  wire [31:0] R14_q,
output  wire [31:0] R15_q,
output  wire [31:0] HI_q_dbg,
output  wire [31:0] LO_q_dbg,
output  wire [31:0] IR_q_dbg,
output  wire [31:0] PC_q_dbg,
output  wire [31:0] MAR_q_dbg,
output  wire [31:0] MDR_q_dbg,
output  wire [31:0] Y_q,
output  wire [63:0] Z_q,
```

```
    // phase 2 debug
    output  wire        CON_q_dbg,
    output  wire [31:0] OutPort_q_dbg,
    output  wire [31:0] InPort_q_dbg,
    output  wire [31:0] RAM_read_data_dbg,
    output  wire [31:0] RAM_dbg_data
);

    // -------- Registers R0..R15 --------
    wire [31:0] R [0:15];
    wire [15:0] Rin_decoded;
    wire [15:0] Rout_decoded;
    wire [3:0]  decoded_reg_idx;
    wire [15:0] Rin_effective;

    assign Rin_effective = Rin | Rin_decoded;

    genvar i;
    generate
      for (i = 0; i < 16; i = i + 1) begin : GEN_REGS
        register32 Ri (
          .clk(clk), .reset(reset),
          .en(Rin_effective[i]),
          .d(BUS),
          .q(R[i])
        );
      end
    endgenerate

    assign R5_q = R[5];
    assign R6_q = R[6];
    assign R2_q = R[2];
    assign R3_q = R[3];
    assign R0_q = R[0];
    assign R1_q = R[1];
    assign R4_q = R[4];
    assign R12_q = R[12];
    assign R7_q = R[7];
    assign R8_q = R[8];
    assign R9_q = R[9];
    assign R10_q = R[10];
    assign R11_q = R[11];
    assign R13_q = R[13];
    assign R14_q = R[14];
    assign R15_q = R[15];

    // -------- Internal special registers --------
    reg [31:0] PC_q;
    reg [31:0] IR_q;
    reg [31:0] MAR_q;
    reg [31:0] MDR_q;
    reg [31:0] HI_q;
    reg [31:0] LO_q;
    reg [31:0] decoded_rout_data;

    wire [31:0] C_sign_extended;
    wire        CON_q;
    wire [31:0] out_port_q;
    wire [31:0] in_port_q;
    wire [31:0] ram_read_data;
    wire [31:0] bus_mux_out;
    wire        decoded_rout_active = |Rout_decoded;
    wire        use_ram_data = (UseRAM === 1'b1);
    wire        write_mem = (Write === 1'b1);
    wire        cout_en = (Cout === 1'b1);
    wire        inportout_en = (InPortout === 1'b1);

    // -------- Y register (operand A latch) --------
    register32 Yreg (
      .clk(clk), .reset(reset),
      .en(Yin),
      .d(BUS),
      .q(Y_q)
```

```
);

// -------- ALU logic (A=Y, B=BUS) --------
wire [63:0] ALU_C;

alu_logic ULOGIC (
  .A(Y_q),
  .B(BUS),
  .op(op),
  .overflow(),
  .C(ALU_C)
);

// IncPC path for T0: Z <- PC + 1 when IncPC is asserted.
wire [63:0] Z_d = IncPC ? {32'b0, (BUS + 32'd1)} : ALU_C;

// -------- Z register --------
register64 Zreg (
  .clk(clk), .reset(reset),
  .en(Zin),
  .d(Z_d),
  .q(Z_q)
);

wire [31:0] Zlow  = Z_q[31:0];
wire [31:0] Zhigh = Z_q[63:32];

assign HI_q_dbg = HI_q;
assign LO_q_dbg = LO_q;
assign IR_q_dbg = IR_q;
assign PC_q_dbg = PC_q;
assign MAR_q_dbg = MAR_q;
assign MDR_q_dbg = MDR_q;
assign CON_q_dbg = CON_q;
assign OutPort_q_dbg = out_port_q;
assign InPort_q_dbg = in_port_q;
assign RAM_read_data_dbg = ram_read_data;

select_encode USELECT (
  .IR(IR_q),
  .Gra(Gra),
  .Grb(Grb),
  .Grc(Grc),
  .Rin_sel(Rin_dec),
  .Rout_sel(Rout_dec),
  .Rin_decoded(Rin_decoded),
  .Rout_decoded(Rout_decoded),
  .selected_reg(decoded_reg_idx)
);

sign_extend_c USEXT (
  .IR(IR_q),
  .Cout(Cout),
  .C_sign_extended(C_sign_extended)
);

con_ff_logic UCON (
  .clk(clk),
  .reset(reset),
  .CONin(CONin),
  .bus_value(BUS),
  .c2(IR_q[20:19]),
  .CON_q(CON_q)
);

io_ports UIO (
  .clk(clk),
  .reset(reset),
  .OutPortin(OutPortin),
  .bus_in(BUS),
  .InPortLoad(InPortLoad),
  .InPort_data(InPort_data),
  .OutPort_q(out_port_q),
  .InPort_q(in_port_q)
```

```
);

ram512x32 #(
  .MEM_INIT_FILE(MEM_INIT_FILE)
) URAM (
  .clk(clk),
  .write_en(write_mem),
  .addr(MAR_q[8:0]),
  .write_data(MDR_q),
  .read_data(ram_read_data),
  .dbg_addr(MemDbgAddr),
  .dbg_data(RAM_dbg_data)
);

always @(*) begin
  case (decoded_reg_idx)
    4'd0:  decoded_rout_data = R[0];
    4'd1:  decoded_rout_data = R[1];
    4'd2:  decoded_rout_data = R[2];
    4'd3:  decoded_rout_data = R[3];
    4'd4:  decoded_rout_data = R[4];
    4'd5:  decoded_rout_data = R[5];
    4'd6:  decoded_rout_data = R[6];
    4'd7:  decoded_rout_data = R[7];
    4'd8:  decoded_rout_data = R[8];
    4'd9:  decoded_rout_data = R[9];
    4'd10: decoded_rout_data = R[10];
    4'd11: decoded_rout_data = R[11];
    4'd12: decoded_rout_data = R[12];
    4'd13: decoded_rout_data = R[13];
    4'd14: decoded_rout_data = R[14];
    4'd15: decoded_rout_data = R[15];
    default: decoded_rout_data = 32'b0;
  endcase
end

wire [31:0] decoded_bus_value = (BAout && (decoded_reg_idx == 4'd0)) ? 32'b0 : decoded_rout_data;

assign BUS = decoded_rout_active ? decoded_bus_value :
             (cout_en ? C_sign_extended :
             (inportout_en ? in_port_q : bus_mux_out));

// Special registers load from control signals.
always @(posedge clk) begin
  if (reset) begin
    PC_q  <= PC;
    IR_q  <= IR;
    MAR_q <= MAR;
    MDR_q <= MDR;
    HI_q  <= HI;
    LO_q  <= LO;
  end else begin
    if (MARin) begin
      MAR_q <= BUS;
    end

    if (MDRin) begin
      MDR_q <= Read ? (use_ram_data ? ram_read_data : Mdatain) : BUS;
    end

    if (IRin) begin
      IR_q <= BUS;
    end

    if (PCin || ((CON_to_PCin === 1'b1) && CON_q)) begin
      PC_q <= BUS;
    end

    if (HIin) begin
      HI_q <= BUS;
    end

    if (LOin) begin
      LO_q <= BUS;
```

```
        end
      end
    end

    bus_mux UBUS (
      .sel(bus_sel),

      .R0(R[0]), .R1(R[1]), .R2(R[2]), .R3(R[3]),
      .R4(R[4]), .R5(R[5]), .R6(R[6]), .R7(R[7]),
      .R8(R[8]), .R9(R[9]), .R10(R[10]), .R11(R[11]),
      .R12(R[12]), .R13(R[13]), .R14(R[14]), .R15(R[15]),

      .PC(PC_q),
      .IR(IR_q),
      .Y(Y_q),
      .MAR(MAR_q),
      .MDR(MDR_q),
      .HI(HI_q),
      .LO(LO_q),

      .Zlow(Zlow),
      .Zhigh(Zhigh),

      .bus(bus_mux_out)
    );

endmodule
```

A.6 Bus Multiplexer (bus_mux.v)

```
module bus_mux (
    input  wire [4:0]  sel,
    input  wire [31:0]  R0, R1, R2, R3, R4, R5, R6, R7, R8, R9, R10, R11, R12, R13, R14, R15,
    input  wire [31:0]  PC, IR, Y, MAR, MDR, HI, LO,
    input  wire [31:0]  Zlow, Zhigh,
    output reg  [31:0]  bus
);
    //Register numbers may need changing
    always @(*) begin
        case (sel)
            5'd0:  bus = R0;
            5'd1:  bus = R1;
            5'd2:  bus = R2;
            5'd3:  bus = R3;
            5'd4:  bus = R4;
            5'd5:  bus = R5;
            5'd6:  bus = R6;
            5'd7:  bus = R7;
            5'd8:  bus = R8;
            5'd9:  bus = R9;
            5'd10: bus = R10;
            5'd11: bus = R11;
            5'd12: bus = R12;
            5'd13: bus = R13;
            5'd14: bus = R14;
            5'd15: bus = R15;

            5'd16: bus = PC;
            5'd17: bus = IR;
            5'd18: bus = Y;
            5'd19: bus = MAR;
            5'd20: bus = MDR;
            5'd21: bus = HI;
            5'd22: bus = LO;
            5'd23: bus = Zlow;
            5'd24: bus = Zhigh;

            default: bus = 32'b0;
        endcase
    end
```

```
endmodule
```

A.7 ALU (alu_logic.v)

```
`include "defines.vh"

module alu_logic (
    input   wire [31:0]     A,
    input   wire [31:0]     B,
    input   wire [3:0]      op,

    output reg          overflow,
    output reg  [63:0]  C
    //Implemented - And(1), Or(2), Addition (CLA)(8), Multiplication(10), Shift Left(3)
                      //Rotate Left(6), Rotate Right(7), Shift Right(4), Shift Right (Signed)(5), negate, not,
Subtraction(9), Division(11),
);

    // Add/Sub wiring
    wire [31:0] B_eff   = (op == `SUBop) ? ~B : B;
    wire        cin_eff = (op == `SUBop) ? 1'b1 : 1'b0;

    wire [31:0] add_sum;
    wire        add_cout;
    wire [63:0] md_sum;

    // Division wiring
    wire [31:0] div_q;
    wire [31:0] div_r;

    wire [4:0] sh = B[4:0];
    wire       add_overflow = (~(A[31] ^ B[31])) & (add_sum[31] ^ A[31]);
    wire       sub_overflow = (A[31] ^ B[31]) & (add_sum[31] ^ A[31]);



    cla32 inst_cla32 (
                .a  (A),
                .b  (B_eff),
                .cin    (cin_eff),
                .sum    (add_sum),
                .cout   (add_cout)
            );

    booth_bit_pair inst_bbp (
                .M(A),
                .Q(B),
                .P(md_sum)
            );


    nonrestoring_div32 u_div (
        .numerator   (A),
        .denominator (B),
        .remainder   (div_r),
        .quotient    (div_q)
    );


    always @(*) begin
        C = 64'b0;
        overflow = 1'b0;

        case (op)

            `ANDop:  C = {32'b0, (A & B)};
            `ORop:   C = {32'b0, (A | B)};
            `SHLop:  C = {32'b0, A << sh};
            `SHRop:  C = {32'b0, A >> sh};
            `SHRAop: C = {32'b0, $signed(A) >>> sh};
            `ROLop:  C = {32'b0, (sh == 0) ? A : ((A << sh) | (A >> (6'd32 - sh)))};
            `RORop:  C = {32'b0, (sh == 0) ? A : ((A >> sh) | (A << (6'd32 - sh)))};
```

```
            `ADDop: begin
                C = {32'b0, add_sum};
                overflow = add_overflow;
            end
            `MULop:  C = {md_sum};
            `NEGop:  C = {32'b0, ~B+1'b1};
            `NOTop:  C = {32'b0, ~B};
            `SUBop: begin
                C = {32'b0, add_sum};
                overflow = sub_overflow;
            end
            `DIVop: C = {div_r, div_q};
            default: begin
                C = 64'b0;
                overflow = 1'b0;
            end

        endcase

    end
endmodule
```

A.8 32-bit CLA Adder (cla32.v)

```
module cla32(
    input  wire [31:0] a,
    input  wire [31:0] b,
    input  wire        cin,
    output wire [31:0] sum,
    output wire        cout
);
    wire [7:0] P, G;      // group propagate/generate from each 4-bit block
    wire [8:0] C;         // carry into each block (C[0]=cin), C[8]=final cout

    assign C[0] = cin;

    genvar k;
    generate
        for (k=0; k<8; k=k+1) begin : BLK
            cla4 inst_cla4 (
                .a   (a[4*k+3 : 4*k]),
                .b   (b[4*k+3 : 4*k]),
                .cin (C[k]),
                .sum (sum[4*k+3 : 4*k]),
                .cout(),        // not needed since we use group P/G + C[k] to form C[k+1] below
                .P   (P[k]), //2 Gate Delays
                .G   (G[k])  //3 Gate Delays
            );
        end
    endgenerate

    // Carry from block to block (hierarchical CLA)
    assign C[1] = G[0] | (P[0] & C[0]); //4 gate delays
    assign C[2] = G[1] | (P[1] & C[1]); //6 gate delays
    assign C[3] = G[2] | (P[2] & C[2]); //8 gate delays
    assign C[4] = G[3] | (P[3] & C[3]); //10 gate delays
    assign C[5] = G[4] | (P[4] & C[4]); //12 gate delays
    assign C[6] = G[5] | (P[5] & C[5]); //14 gate delays
    assign C[7] = G[6] | (P[6] & C[6]); //16 gate delays
    assign C[8] = G[7] | (P[7] & C[7]); //18 gate delays

    assign cout = C[8];
endmodule
```

A.9 4-bit CLA Block (cla4.v)

```
module cla4(
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire       cin,
```

```
    output wire [3:0] sum,
    output wire       cout,
    output wire       P,   // group propagate
    output wire       G    // group generate
);
    wire [3:0] p, g;
    wire c1, c2, c3, c4;

    //First Gate Delay
    assign p = a ^ b; //XOR
    assign g = a & b;

    //Second(&) and Third(|) Gate Delay
    assign c1 = g[0] | (p[0] & cin);
    assign c2 = g[1] | (p[1] & g[0]) | (p[1] & p[0] & cin);
    assign c3 = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]) | (p[2] & p[1] & p[0] & cin);
    assign c4 = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1])
                     | (p[3] & p[2] & p[1] & g[0])
                     | (p[3] & p[2] & p[1] & p[0] & cin);

    //Fourth Gate Delay
    assign sum[0] = p[0] ^ cin;
    assign sum[1] = p[1] ^ c1;
    assign sum[2] = p[2] ^ c2;
    assign sum[3] = p[3] ^ c3;

    assign cout = c4;

    // Group P/G for building 32-bit hierarchy
    assign P = p[3] & p[2] & p[1] & p[0]; //2 Delays
    assign G = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) | (p[3] & p[2] & p[1] & g[0]); //3 Delays
endmodule
```

A.10 Booth Multiplier (booth_bit_pair.v)

```
module booth_bit_pair #(
    parameter N = 32
)(
    input  wire signed [N-1:0] M,   // multiplicand
    input  wire signed [N-1:0] Q,   // multiplier
    output reg  signed [2*N-1:0] P  // product
);

    integer i;

    reg signed [2*N-1:0] M_ext;
    reg signed [2*N-1:0] pp;        // current partial product (unshifted)
    reg [2:0] booth_bits;

    always @(*) begin
        M_ext = {{N{M[N-1]}}, M};   // sign-extend M to 2N
        P     = {2*N{1'b0}};

        //Bit Pair: (q[2i+1], q[2i], q[2i-1]) with q[-1] = 0
        for (i = 0; i < N/2; i = i + 1) begin
            booth_bits[0] = (i == 0) ? 1'b0 : Q[2*i - 1];

            booth_bits[1] = Q[2*i];

            booth_bits[2] = ((2*i + 1) < N) ? Q[2*i + 1] : Q[N-1];

            // Decode to pp in {-2M, -M, 0, +M, +2M}
            if ((booth_bits == 3'b000) || (booth_bits == 3'b111)) begin
                pp = {2*N{1'b0}};
            end else if ((booth_bits == 3'b001) || (booth_bits == 3'b010)) begin
                pp = M_ext;
            end else if (booth_bits == 3'b011) begin
                pp = M_ext <<< 1;
            end else if (booth_bits == 3'b100) begin
                pp = -(M_ext <<< 1);
            end else if ((booth_bits == 3'b101) || (booth_bits == 3'b110)) begin
                pp = -M_ext;
```

```
            end else begin
                pp = {2*N{1'b0}};
            end

            // Shift by 2i (shifted for the )
            P = P + (pp <<< (2*i));
        end
    end

endmodule
```

A.11 Non-Restoring Divider (nonrestoring_div32.v)

```
module nonrestoring_div32(
    input  wire [31:0] numerator,
    input  wire [31:0] denominator,
    output reg  [31:0] remainder,
    output reg  [31:0] quotient
);

    reg  signed [32:0] a;
    reg         [31:0] q;
    reg  signed [32:0] m;

    reg num_neg, den_neg;

    integer i;

    reg [31:0] num_mag, den_mag;

    always @(*) begin
        // Defaults prevent inferred latches in combinational implementation.
        quotient  = 32'b0;
        remainder = 32'b0;
        a         = 33'sd0;
        q         = 32'b0;
        m         = 33'sd0;
        num_mag   = 32'b0;
        den_mag   = 32'b0;
        i         = 0;

        num_neg = ($signed(numerator)   < 0);
        den_neg = ($signed(denominator) < 0);

        if (denominator == 32'b0) begin
            quotient  = 32'hFFFF_FFFF;
            remainder = numerator;
        end else begin
            num_mag = num_neg ? (~numerator + 1'b1) : numerator;
            den_mag = den_neg ? (~denominator + 1'b1) : denominator;

            a = 33'sd0;
            q = num_mag;
            m = {1'b0, den_mag};

            for (i = 0; i < 32; i = i + 1) begin
                a = {a[31:0], q[31]};
                q = q << 1;

                if (a >= 0)
                    a = a - m;
                else
                    a = a + m;

                q[0] = (a >= 0);
            end

            if (a < 0) begin
                a = a + m;
            end

            if (num_neg ^ den_neg) begin
```

```
                quotient = ~q + 1'b1;
            end else begin
                quotient = q;
            end

            if (num_neg) begin
                remainder = ~a[31:0] + 1'b1;
            end else begin
                remainder = a[31:0];
            end
        end
    end

endmodule
```

A.12 Select and Encode (select_encode.v)

```
module select_encode (
    input  wire [31:0] IR,
    input  wire        Gra,
    input  wire        Grb,
    input  wire        Grc,
    input  wire        Rin_sel,
    input  wire        Rout_sel,
    output reg  [15:0] Rin_decoded,
    output reg  [15:0] Rout_decoded,
    output reg  [3:0]  selected_reg
);
    reg [15:0] onehot;

    always @(*) begin
        if (Gra) begin
            selected_reg = IR[26:23];
        end else if (Grb) begin
            selected_reg = IR[22:19];
        end else if (Grc) begin
            selected_reg = IR[18:15];
        end else begin
            selected_reg = 4'd0;
        end

        onehot = 16'b0;
        onehot[selected_reg] = 1'b1;

        if (Rin_sel === 1'b1) begin
            Rin_decoded = onehot;
        end else begin
            Rin_decoded = 16'b0;
        end

        if (Rout_sel === 1'b1) begin
            Rout_decoded = onehot;
        end else begin
            Rout_decoded = 16'b0;
        end
    end
endmodule
```

A.13 Sign Extension (sign_extend_c.v)

```
module sign_extend_c (
    input  wire [31:0] IR,
    input  wire        Cout,
    output wire [31:0] C_sign_extended
);
    wire [31:0] sext_c = {{13{IR[18]}}, IR[18:0]};
    assign C_sign_extended = Cout ? sext_c : 32'b0;
endmodule
```

A.14 Condition Logic (con_ff_logic.v)

```
module con_ff_logic (
    input  wire        clk,
    input  wire        reset,
    input  wire        CONin,
    input  wire [31:0] bus_value,
    input  wire [1:0]  c2,
    output reg         CON_q
);
    reg cond_eval;

    always @(*) begin
        case (c2)
            2'b00: cond_eval = (bus_value == 32'b0);                              // brzr
            2'b01: cond_eval = (bus_value != 32'b0);                              // brnz
            2'b10: cond_eval = (~bus_value[31]) & (bus_value != 32'b0);           // brpl
            2'b11: cond_eval = bus_value[31];                                      // brmi
            default: cond_eval = 1'b0;
        endcase
    end

    always @(posedge clk) begin
        if (reset) begin
            CON_q <= 1'b0;
        end else if (CONin) begin
            CON_q <= cond_eval;
        end
    end
endmodule
```

A.15 IO Ports (io_ports.v)

```
module io_ports (
    input  wire        clk,
    input  wire        reset,
    input  wire        OutPortin,
    input  wire [31:0] bus_in,
    input  wire        InPortLoad,
    input  wire [31:0] InPort_data,
    output reg  [31:0] OutPort_q,
    output reg  [31:0] InPort_q
);
    always @(posedge clk) begin
        if (reset) begin
            OutPort_q <= 32'b0;
            InPort_q  <= 32'b0;
        end else begin
            if (OutPortin) begin
                OutPort_q <= bus_in;
            end
            if (InPortLoad) begin
                InPort_q <= InPort_data;
            end
        end
    end
endmodule
```

A.16 RAM (ram512x32.v)

```
module ram512x32 #(
    parameter MEM_INIT_FILE = ""
) (
    input  wire        clk,
    input  wire        write_en,
    input  wire [8:0]  addr,
    input  wire [31:0] write_data,
    output wire [31:0] read_data,
    input  wire [8:0]  dbg_addr,
    output wire [31:0] dbg_data
```

```
);
    reg [31:0] mem [0:511];
    integer i;

    initial begin
        for (i = 0; i < 512; i = i + 1) begin
            mem[i] = 32'b0;
        end
        if (MEM_INIT_FILE != "") begin
            $readmemh(MEM_INIT_FILE, mem);
        end
    end

    always @(posedge clk) begin
        if (write_en) begin
            mem[addr] <= write_data;
        end
    end

    assign read_data = mem[addr];
    assign dbg_data  = mem[dbg_addr];
endmodule
```

A.17 32-bit Register (register32.v)

```
// register32.v
module register32 (
    input  wire        clk,
    input  wire        reset,   // set to 0 on reset
    input  wire        en,      // load enable
    input  wire [31:0] d,
    output reg  [31:0] q
);

    // Synchronous reset
    always @(posedge clk) begin
        if (reset) begin
            q <= 32'b0;
        end else if (en) begin
            q <= d;
        end
    end

endmodule
```

A.18 64-bit Register (register64.v)

```
// register64.v
module register64 (
    input  wire        clk,
    input  wire        reset,
    input  wire        en,
    input  wire [63:0] d,
    output reg  [63:0] q
);

    always @(posedge clk) begin
        if (reset) begin
            q <= 64'b0;
        end else if (en) begin
            q <= d;
        end
    end

endmodule
```

A.19 Seven Segment Decoder (seven_seg_hex.v)

```
module seven_seg_hex (
    input  wire [3:0] hex,
    output reg  [6:0] seg_n
);
    always @(*) begin
        case (hex)
            4'h0: seg_n = 7'b1000000;
            4'h1: seg_n = 7'b1111001;
            4'h2: seg_n = 7'b0100100;
            4'h3: seg_n = 7'b0110000;
            4'h4: seg_n = 7'b0011001;
            4'h5: seg_n = 7'b0010010;
            4'h6: seg_n = 7'b0000010;
            4'h7: seg_n = 7'b1111000;
            4'h8: seg_n = 7'b0000000;
            4'h9: seg_n = 7'b0010000;
            4'hA: seg_n = 7'b0001000;
            4'hB: seg_n = 7'b0000011;
            4'hC: seg_n = 7'b1000110;
            4'hD: seg_n = 7'b0100001;
            4'hE: seg_n = 7'b0000110;
            4'hF: seg_n = 7'b0001110;
            default: seg_n = 7'b1111111;
        endcase
    end
endmodule
```

Appendix B: Testbenches and Test Programs

B.1 Phase 3 Testbench (tb_phase3_cpu.v)

```
`timescale 1ns/1ps

module tb_phase3_cpu;
  reg clk = 1'b0;
  always #5 clk = ~clk;

  reg         reset;
  reg         stop;
  reg  [31:0] in_port_data;
  reg  [8:0]  mem_dbg_addr;

  wire        run;
  wire [31:0] bus_dbg;
  wire [31:0] ir_dbg;
  wire [31:0] pc_dbg;
  wire [31:0] mar_dbg;
  wire [31:0] mdr_dbg;
  wire [31:0] hi_dbg;
  wire [31:0] lo_dbg;
  wire [31:0] y_dbg;
  wire [63:0] z_dbg;
  wire        con_dbg;
  wire [31:0] out_port_dbg;
  wire [31:0] in_port_dbg;
  wire [31:0] ram_read_data_dbg;
  wire [31:0] ram_dbg_data;
  wire [31:0] r0_dbg;
  wire [31:0] r1_dbg;
  wire [31:0] r2_dbg;
  wire [31:0] r3_dbg;
  wire [31:0] r4_dbg;
  wire [31:0] r5_dbg;
  wire [31:0] r6_dbg;
  wire [31:0] r7_dbg;
  wire [31:0] r8_dbg;
  wire [31:0] r9_dbg;
```

```
wire [31:0] r10_dbg;
wire [31:0] r11_dbg;
wire [31:0] r12_dbg;
wire [31:0] r13_dbg;
wire [31:0] r14_dbg;
wire [31:0] r15_dbg;

localparam [31:0] IR_BRMI            = 32'hAA980003;
localparam [31:0] IR_BRPL            = 32'hA8900002;
localparam [31:0] IR_LDI_AFTER_BRMI  = 32'h8AA80005;
localparam [31:0] IR_SKIP_12         = 32'h89A80007;
localparam [31:0] IR_SKIP_13         = 32'h8B9FFFFC;
localparam [31:0] IR_JAL             = 32'h9D000000;
localparam [31:0] IR_JR              = 32'hA6000000;
localparam [31:0] IR_HALT            = 32'hD8000000;

integer failures;
integer cycles;

reg seen_brmi;
reg seen_brpl;
reg seen_after_brmi;
reg seen_skip_12;
reg seen_skip_13;
reg seen_jal;
reg seen_jr;
reg seen_jal_effect;

cpu_phase3 #(
  .MEM_INIT_FILE("tb/phase3_mem_init.hex")
) dut (
  .clk(clk),
  .reset(reset),
  .stop(stop),
  .in_port_data(in_port_data),
  .mem_dbg_addr(mem_dbg_addr),
  .run(run),
  .bus_dbg(bus_dbg),
  .ir_dbg(ir_dbg),
  .pc_dbg(pc_dbg),
  .mar_dbg(mar_dbg),
  .mdr_dbg(mdr_dbg),
  .hi_dbg(hi_dbg),
  .lo_dbg(lo_dbg),
  .y_dbg(y_dbg),
  .z_dbg(z_dbg),
  .con_dbg(con_dbg),
  .out_port_dbg(out_port_dbg),
  .in_port_dbg(in_port_dbg),
  .ram_read_data_dbg(ram_read_data_dbg),
  .ram_dbg_data(ram_dbg_data),
  .r0_dbg(r0_dbg),
  .r1_dbg(r1_dbg),
  .r2_dbg(r2_dbg),
  .r3_dbg(r3_dbg),
  .r4_dbg(r4_dbg),
  .r5_dbg(r5_dbg),
  .r6_dbg(r6_dbg),
  .r7_dbg(r7_dbg),
  .r8_dbg(r8_dbg),
  .r9_dbg(r9_dbg),
  .r10_dbg(r10_dbg),
  .r11_dbg(r11_dbg),
  .r12_dbg(r12_dbg),
  .r13_dbg(r13_dbg),
  .r14_dbg(r14_dbg),
  .r15_dbg(r15_dbg)
);

task check_eq;
  input [31:0] got;
  input [31:0] exp;
  input [255:0] msg;
  begin
```

```
    if (got !== exp) begin
      failures = failures + 1;
      $display("FAIL: %0s got=%h exp=%h", msg, got, exp);
    end else begin
      $display("PASS: %0s value=%h", msg, got);
    end
  end
endtask

task check_mem;
  input [8:0] addr;
  input [31:0] exp;
  input [255:0] msg;
  begin
    mem_dbg_addr = addr;
    #1;
    check_eq(ram_dbg_data, exp, msg);
  end
endtask

always @(posedge clk) begin
  if (!reset) begin
    if (ir_dbg == IR_BRMI) begin
      seen_brmi <= 1'b1;
    end
    if (ir_dbg == IR_BRPL) begin
      seen_brpl <= 1'b1;
    end
    if (ir_dbg == IR_LDI_AFTER_BRMI) begin
      seen_after_brmi <= 1'b1;
    end
    if (ir_dbg == IR_SKIP_12) begin
      seen_skip_12 <= 1'b1;
    end
    if (ir_dbg == IR_SKIP_13) begin
      seen_skip_13 <= 1'b1;
    end
    if (ir_dbg == IR_JAL) begin
      seen_jal <= 1'b1;
    end
    if (ir_dbg == IR_JR) begin
      seen_jr <= 1'b1;
    end
    if ((r12_dbg == 32'h00000029) && (pc_dbg == 32'h000000B2)) begin
      seen_jal_effect <= 1'b1;
    end
  end
end

initial begin
  $dumpfile("phase3_cpu.vcd");
  $dumpvars(0, tb_phase3_cpu);

  failures = 0;
  cycles = 0;

  seen_brmi = 1'b0;
  seen_brpl = 1'b0;
  seen_after_brmi = 1'b0;
  seen_skip_12 = 1'b0;
  seen_skip_13 = 1'b0;
  seen_jal = 1'b0;
  seen_jr = 1'b0;
  seen_jal_effect = 1'b0;

  stop = 1'b0;
  in_port_data = 32'b0;
  mem_dbg_addr = 9'd0;

  reset = 1'b1;
  @(posedge clk);
  @(posedge clk);
  reset = 1'b0;
```

```
while (run && (cycles < 5000)) begin
  @(posedge clk);
  cycles = cycles + 1;
end

if (run) begin
  failures = failures + 1;
  $display("FAIL: timeout waiting for halt");
end else begin
  $display("PASS: halted in %0d cycles", cycles);
end

// Final architectural state checks.
check_eq(ir_dbg, IR_HALT, "halt IR latched");
check_eq(r0_dbg, 32'h00000614, "R0 final");
check_eq(r1_dbg, 32'h00000000, "R1 final");
check_eq(r2_dbg, 32'h00000004, "R2 final");
check_eq(r3_dbg, 32'h00000019, "R3 final");
check_eq(r4_dbg, 32'h00006800, "R4 final");
check_eq(r5_dbg, 32'h00000680, "R5 final");
check_eq(r6_dbg, 32'h000000AF, "R6 final");
check_eq(r7_dbg, 32'h00000007, "R7 final");
check_eq(r8_dbg, 32'h00000009, "R8 final");
check_eq(r9_dbg, 32'h00000015, "R9 final");
check_eq(r10_dbg, 32'h000000B2, "R10 final");
check_eq(r11_dbg, 32'h00000005, "R11 final");
check_eq(r12_dbg, 32'h00000029, "R12 return address");
check_eq(r13_dbg, 32'h00000010, "R13 final");
check_eq(r14_dbg, 32'h000000AB, "R14 final");
check_eq(r15_dbg, 32'h00000000, "R15 final");
check_eq(hi_dbg, 32'h00000004, "HI final");
check_eq(lo_dbg, 32'h00000003, "LO final");

check_mem(9'h0A3, 32'h00000008, "mem[0xA3] final");
check_mem(9'h089, 32'h0000006C, "mem[0x89] final");

// Control-flow evidence checks.
if (!seen_brmi) begin
  failures = failures + 1;
  $display("FAIL: brmi instruction was not observed");
end else begin
  $display("PASS: brmi observed");
end

if (!seen_after_brmi) begin
  failures = failures + 1;
  $display("FAIL: brmi not-taken path evidence missing (next instruction not observed)");
end else begin
  $display("PASS: brmi not-taken path observed");
end

if (!seen_brpl) begin
  failures = failures + 1;
  $display("FAIL: brpl instruction was not observed");
end else begin
  $display("PASS: brpl observed");
end

if (seen_skip_12 || seen_skip_13) begin
  failures = failures + 1;
  $display("FAIL: brpl-taken path violated (skipped instructions executed)");
end else begin
  $display("PASS: brpl taken path observed (skipped instructions not executed)");
end

if (!seen_jal || !seen_jal_effect || !seen_jr) begin
  failures = failures + 1;
  $display("FAIL: jal/jr control-flow evidence missing jal=%0d jal_effect=%0d jr=%0d",
           seen_jal, seen_jal_effect, seen_jr);
end else begin
  $display("PASS: jal/jr control-flow observed");
end

if (failures == 0) begin
```

```
      $display("PASS PHASE3 CPU SUITE");
    end else begin
      $display("FAIL PHASE3 CPU SUITE: failures=%0d", failures);
      $fatal;
    end

    $finish;
  end

endmodule
```

B.2 Phase 4 Testbench (tb_phase4_cpu.v)

```
`timescale 1ns/1ps

module tb_phase4_cpu;
  reg clk = 1'b0;
  always #5 clk = ~clk;

  reg         reset;
  reg         stop;
  reg  [31:0] in_port_data;
  reg  [8:0]  mem_dbg_addr;

  wire        run;
  wire [31:0] ir_dbg;
  wire [31:0] hi_dbg;
  wire [31:0] lo_dbg;
  wire [31:0] out_port_dbg;
  wire [31:0] ram_dbg_data;
  wire [31:0] r2_dbg;
  wire [31:0] r3_dbg;
  wire [31:0] r5_dbg;
  wire [31:0] r6_dbg;
  wire [31:0] r7_dbg;
  wire [31:0] r12_dbg;

  integer failures;
  integer cycles;
  integer out_count;
  integer i;

  reg [31:0] last_out;
  reg [31:0] expected_out [0:40];

  cpu_phase3 #(
    .MEM_INIT_FILE("tb/phase4_mem_init.hex")
  ) dut (
    .clk(clk),
    .reset(reset),
    .stop(stop),
    .in_port_data(in_port_data),
    .mem_dbg_addr(mem_dbg_addr),
    .run(run),
    .bus_dbg(),
    .ir_dbg(ir_dbg),
    .pc_dbg(),
    .mar_dbg(),
    .mdr_dbg(),
    .hi_dbg(hi_dbg),
    .lo_dbg(lo_dbg),
    .y_dbg(),
    .z_dbg(),
    .con_dbg(),
    .out_port_dbg(out_port_dbg),
    .in_port_dbg(),
    .ram_read_data_dbg(),
    .ram_dbg_data(ram_dbg_data),
    .r0_dbg(),
    .r1_dbg(),
    .r2_dbg(r2_dbg),
    .r3_dbg(r3_dbg),
```

```
  .r4_dbg(),
  .r5_dbg(r5_dbg),
  .r6_dbg(r6_dbg),
  .r7_dbg(r7_dbg),
  .r8_dbg(),
  .r9_dbg(),
  .r10_dbg(),
  .r11_dbg(),
  .r12_dbg(r12_dbg),
  .r13_dbg(),
  .r14_dbg(),
  .r15_dbg()
);

task check_eq;
  input [31:0] got;
  input [31:0] exp;
  input [255:0] msg;
  begin
    if (got !== exp) begin
      failures = failures + 1;
      $display("FAIL: %0s got=%h exp=%h", msg, got, exp);
    end else begin
      $display("PASS: %0s value=%h", msg, got);
    end
  end
endtask

task check_mem;
  input [8:0] addr;
  input [31:0] exp;
  input [255:0] msg;
  begin
    mem_dbg_addr = addr;
    #1;
    check_eq(ram_dbg_data, exp, msg);
  end
endtask

initial begin
  for (i = 0; i < 5; i = i + 1) begin
    expected_out[(i * 8) + 0] = 32'h000000E0;
    expected_out[(i * 8) + 1] = 32'h00000070;
    expected_out[(i * 8) + 2] = 32'h00000038;
    expected_out[(i * 8) + 3] = 32'h0000001C;
    expected_out[(i * 8) + 4] = 32'h0000000E;
    expected_out[(i * 8) + 5] = 32'h00000007;
    expected_out[(i * 8) + 6] = 32'h00000003;
    expected_out[(i * 8) + 7] = 32'h00000001;
  end
  expected_out[40] = 32'h00000063;

  failures = 0;
  cycles = 0;
  out_count = 0;
  last_out = 32'b0;

  stop = 1'b0;
  in_port_data = 32'h000000E0;
  mem_dbg_addr = 9'd0;

  reset = 1'b1;
  @(posedge clk);
  @(posedge clk);
  reset = 1'b0;

  // Keep the board image exact, but shrink the delay loop for regression speed.
  dut.UDATAPATH.URAM.mem[9'h088] = 32'h00000004;

  while (run && (cycles < 50000)) begin
    @(posedge clk);
    #1;
    cycles = cycles + 1;
```

```
      if (out_port_dbg !== last_out) begin
        if (out_count > 40) begin
          failures = failures + 1;
          $display("FAIL: unexpected extra output value=%h at index=%0d", out_port_dbg, out_count);
        end else if (out_port_dbg !== expected_out[out_count]) begin
          failures = failures + 1;
          $display("FAIL: output[%0d] got=%h exp=%h", out_count, out_port_dbg, expected_out[out_count]);
        end else begin
          $display("PASS: output[%0d] value=%h", out_count, out_port_dbg);
        end

        last_out = out_port_dbg;
        out_count = out_count + 1;
      end
    end

    if (run) begin
      failures = failures + 1;
      $display("FAIL: timeout waiting for halt");
    end else begin
      $display("PASS: halted in %0d cycles", cycles);
    end

    if (out_count !== 41) begin
      failures = failures + 1;
      $display("FAIL: expected 41 output updates, observed %0d", out_count);
    end else begin
      $display("PASS: observed all 41 output updates");
    end

    check_eq(ir_dbg, 32'hD8000000, "halt IR latched");
    check_eq(r2_dbg, 32'h00000000, "R2 loop counter exhausted");
    check_eq(r3_dbg, 32'h0000002E, "R3 loop address");
    check_eq(r5_dbg, 32'h00000001, "R5 shift amount");
    check_eq(r6_dbg, 32'h00000063, "R6 final display value");
    check_eq(r7_dbg, 32'h00000000, "R7 delay counter drained");
    check_eq(r12_dbg, 32'h00000029, "R12 preserved return address");
    check_eq(hi_dbg, 32'h00000004, "HI final");
    check_eq(lo_dbg, 32'h00000003, "LO final");
    check_eq(out_port_dbg, 32'h00000063, "output port final");

    check_mem(9'h077, 32'h000000E0, "mem[0x77] saved input");
    check_mem(9'h089, 32'h0000006C, "mem[0x89] final");
    check_mem(9'h0A3, 32'h00000008, "mem[0xA3] final");

    if (failures == 0) begin
      $display("PASS PHASE4 CPU SUITE");
    end else begin
      $display("FAIL PHASE4 CPU SUITE: failures=%0d", failures);
      $fatal;
    end

    $finish;
  end
endmodule
```

B.3 Phase 3 Memory Init (phase3_mem_init.hex)

```
@000
8A800043
8AA80006
82000089
8A200004
8027FFF8
89000004
8A800087
AA980003
8AA80005
80AFFFFD
D0000000
A8900002
89A80007
```

```
8B9FFFFC
03A90000
48880003
70880000
78880000
5088000F
3A010000
58A00005
2A090000
22A90000
928000A3
42810000
1B900000
12280000
93A00089
082B8000
32290000
8B800007
89800019
69B80000
C0800000
CB000000
61B80000
8C380002
8C9FFFFC
8D300003
8D880005
9D000000
D8000000
@089
000000A7
@0A3
00000068
@0B2
07450000
0ECD8000
0F768000
A6000000
```

B.4 Phase 4 Memory Init (phase4_mem_init.hex)

```
@000
8A800043
8AA80006
82000089
8A200004
8027FFF8
89000004
8A800087
AA980003
8AA80005
80AFFFFD
D0000000
A8900002
89A80007
8B9FFFFC
03A90000
48880003
70880000
78880000
5088000F
3A010000
58A00005
2A090000
22A90000
928000A3
42810000
1B900000
12280000
93A00089
082B8000
32290000
```

```
8B800007
89800019
69B80000
C0800000
CB000000
61B80000
8C380002
8C9FFFFC
8D300003
8D880005
9D000000
B3000000
93000077
8980002E
8A800001
89000028
BB000000
8917FFFF
A9000008
83800088
8BBFFFFF
D0000000
AB8FFFFD
23328000
AB0FFFF7
83000077
A1800000
8B000063
BB000000
D8000000
@077
00000000
@088
0000FFFF
000000A7
@0A3
00000068
@0B2
07450000
0ECD8000
0F768000
A6000000
```

Appendix C: Functional Simulation Results

The Phase 3 simulation completes in 310 cycles with all 27 verification checks passing. The Phase 4 simulation completes in 5269 cycles with all 55 verification checks passing. The transcript output for the Phase 3 test program is shown below.

```
# PASS: halted in 310 cycles
# PASS: halt IR latched value=d8000000
# PASS: R0 final value=00000614
# PASS: R1 final value=00000000
# PASS: R2 final value=00000004
# PASS: R3 final value=00000019
# PASS: R4 final value=00006800
# PASS: R5 final value=00000680
# PASS: R6 final value=000000af
# PASS: R7 final value=00000007
# PASS: R8 final value=00000009
# PASS: R9 final value=00000015
# PASS: R10 final value=000000b2
# PASS: R11 final value=00000005
# PASS: R12 return address value=00000029
# PASS: R13 final value=00000010
# PASS: R14 final value=000000ab
# PASS: R15 final value=00000000
# PASS: HI final value=00000004
# PASS: LO final value=00000003
# PASS: mem[0xA3] final value=00000008
# PASS: mem[0x89] final value=0000006c
# PASS: brmi observed
# PASS: brmi not-taken path observed
# PASS: brpl observed
# PASS: brpl taken path observed (skipped instructions not executed)
# PASS: jal/jr control-flow observed
# PASS PHASE3 CPU SUITE
```

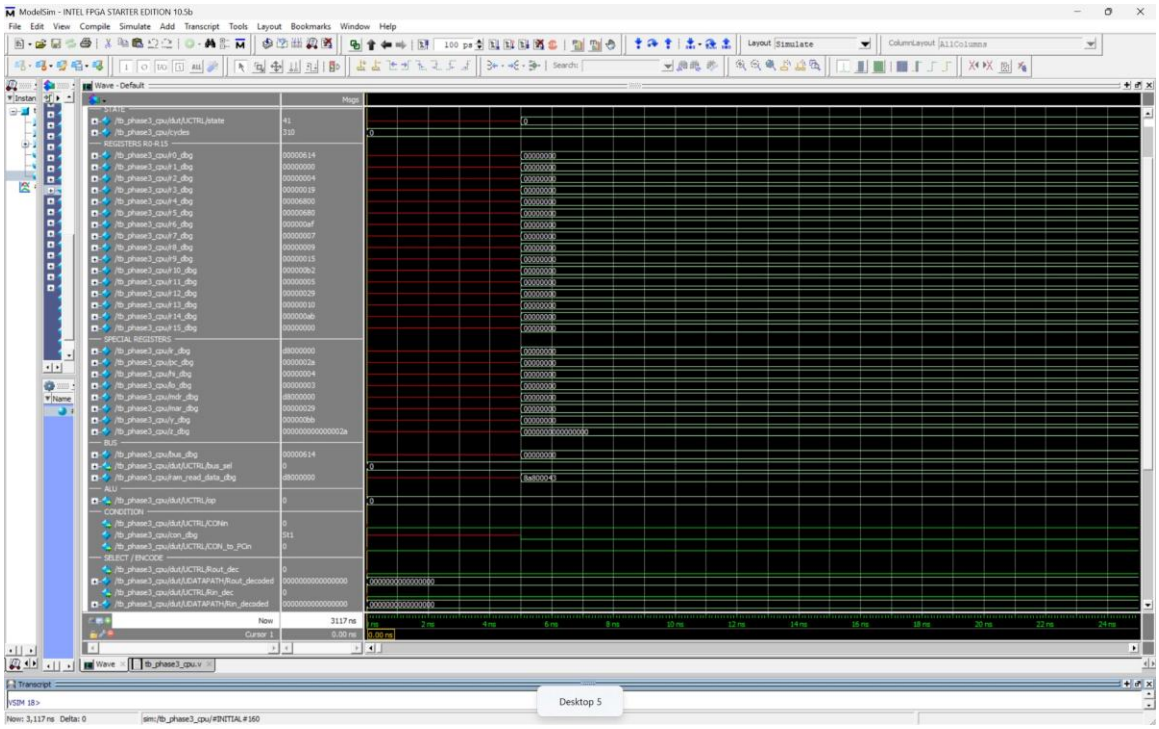
Phase 3 simulation transcript: all 27 PASS checks.

The following table summarizes the memory contents before and after execution for both Phase 3 and Phase 4 test programs.

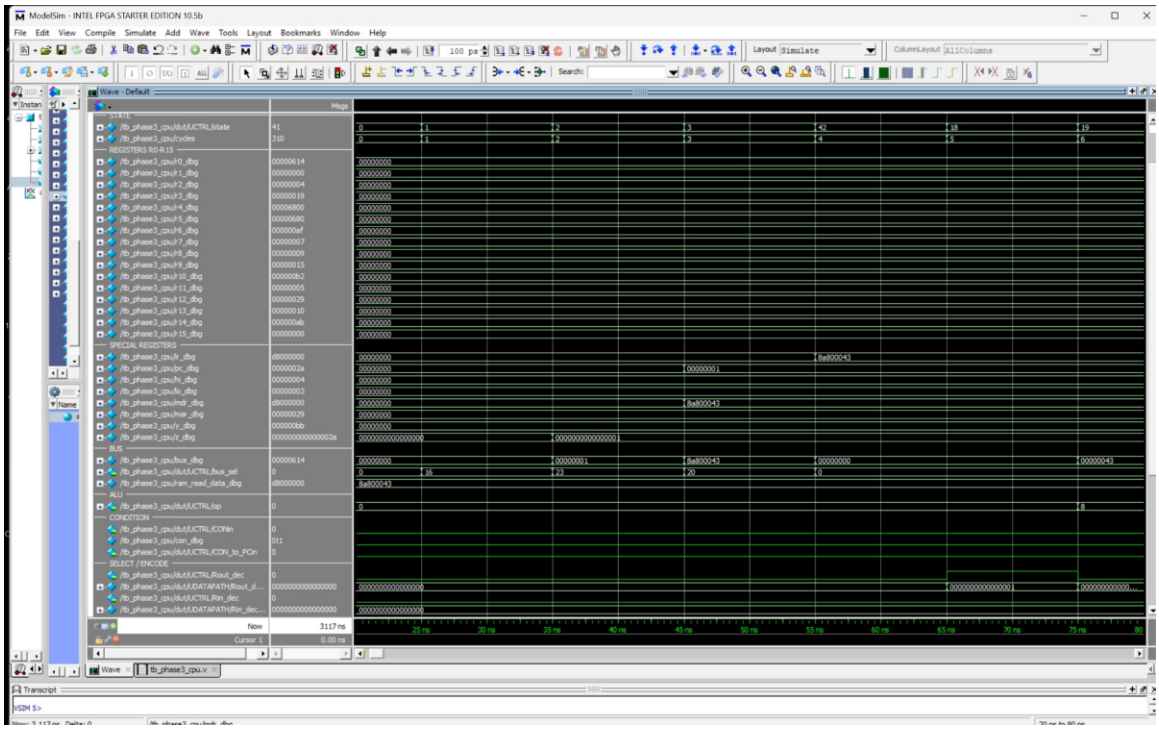
Address	Phase 3 Before	Phase 3 After	Phase 4 After
0x77	-	-	0x000000E0
0x89	0x000000A7	0x0000006C	0x0000006C
0xA3	0x00000068	0x00000008	0x00000008
0x88	-	-	0x0000FFFF

Appendix D: Phase 4 Waveform Screenshots

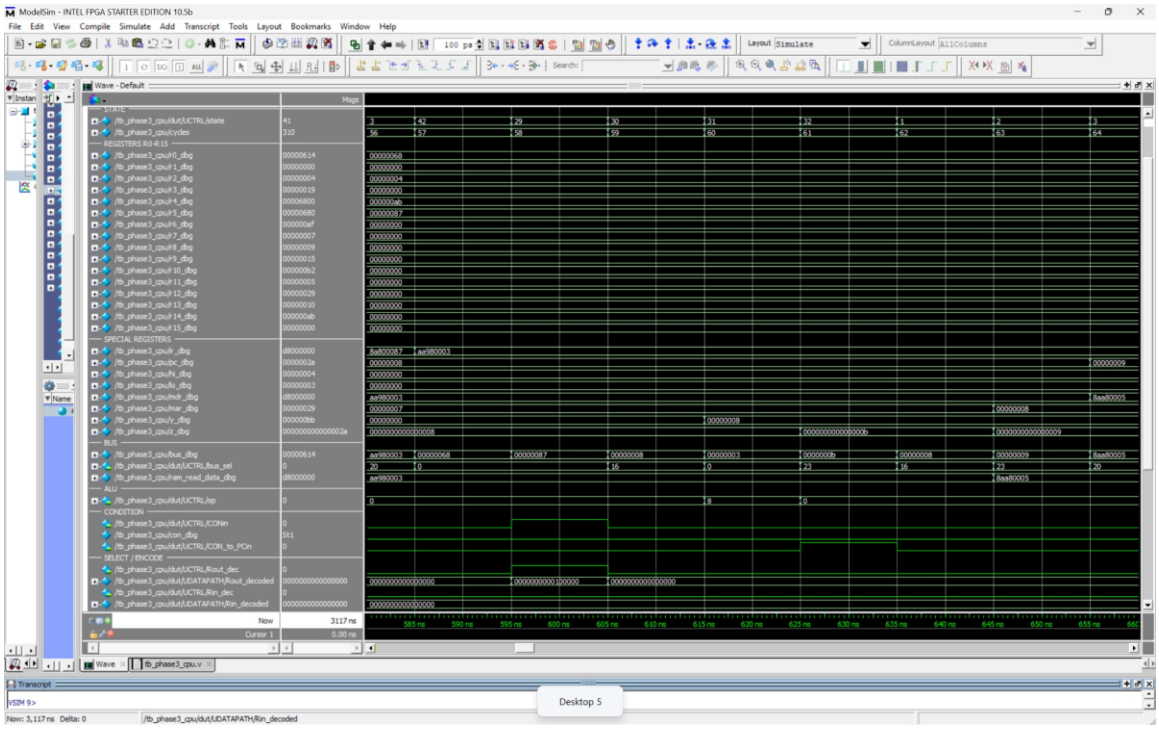
The following screenshots show the functional simulation waveforms for both Phase 3 and Phase 4 test programs. The Phase 3 screenshots (D.1 through D.8) demonstrate individual instruction behavior. The Phase 4 screenshots (D.10 and D.11) show the full Phase 4 simulation and the output display loop.



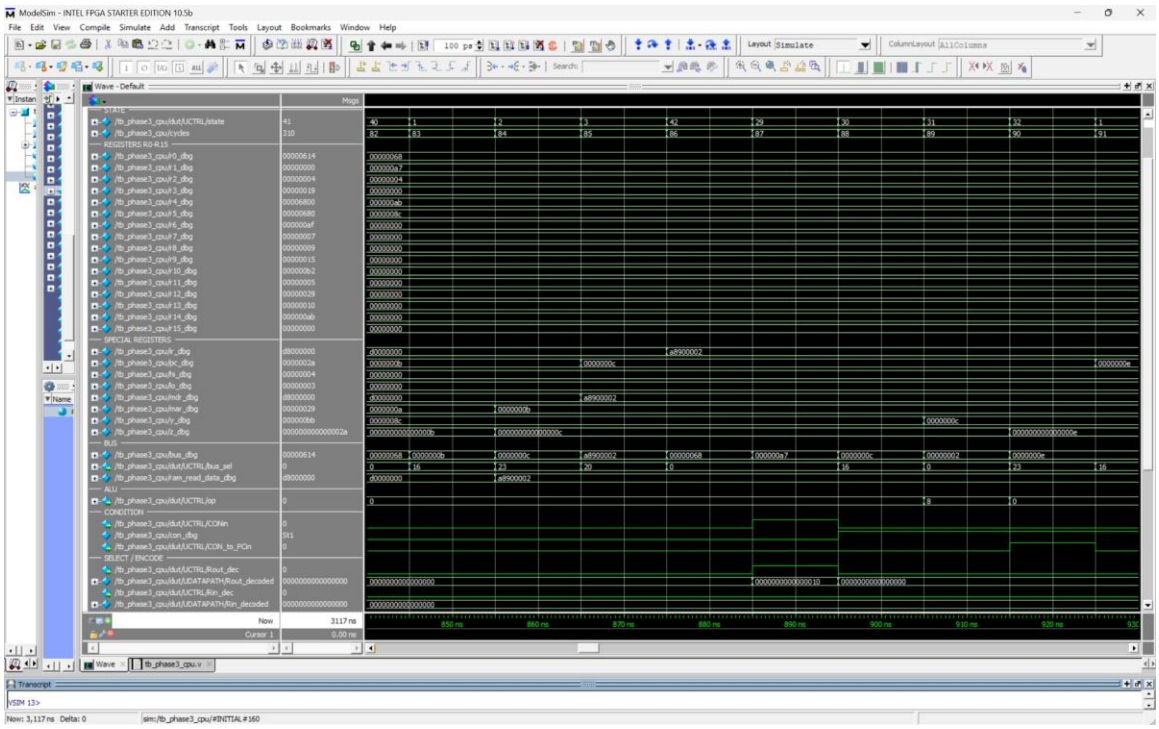
D.1 Initial register state at t=0. All registers are zero.



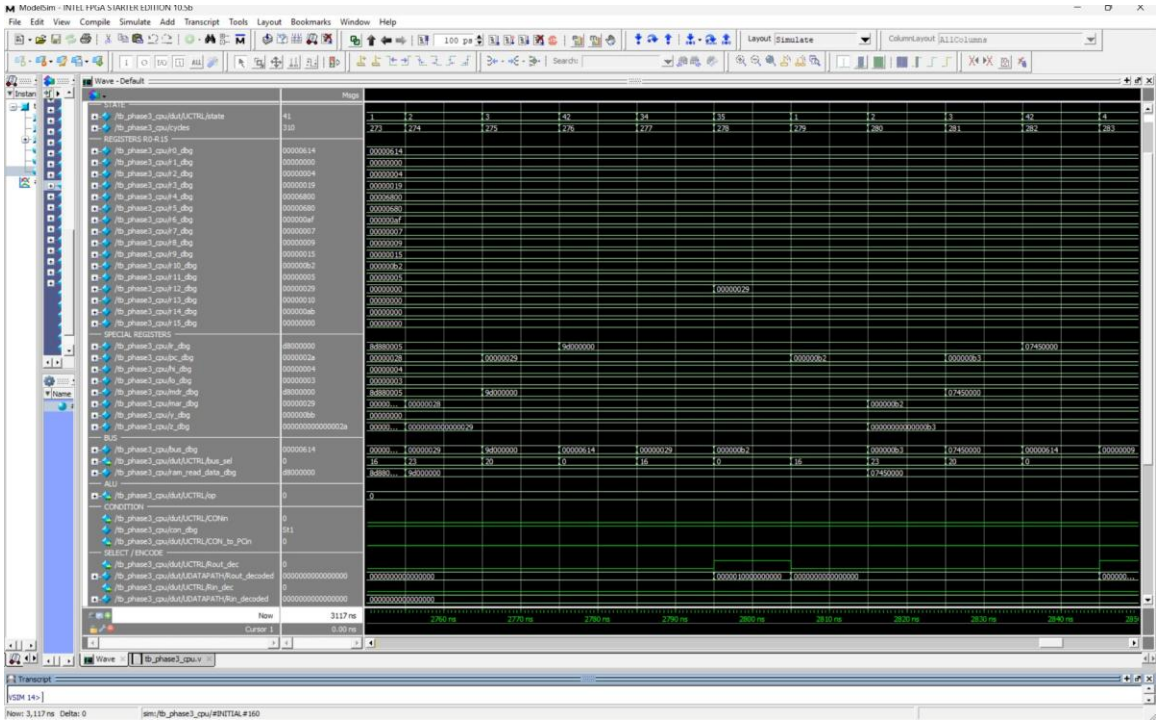
D.2 Fetch cycle: state 1, 2, 3, 42. PC increments, IR loads first instruction.



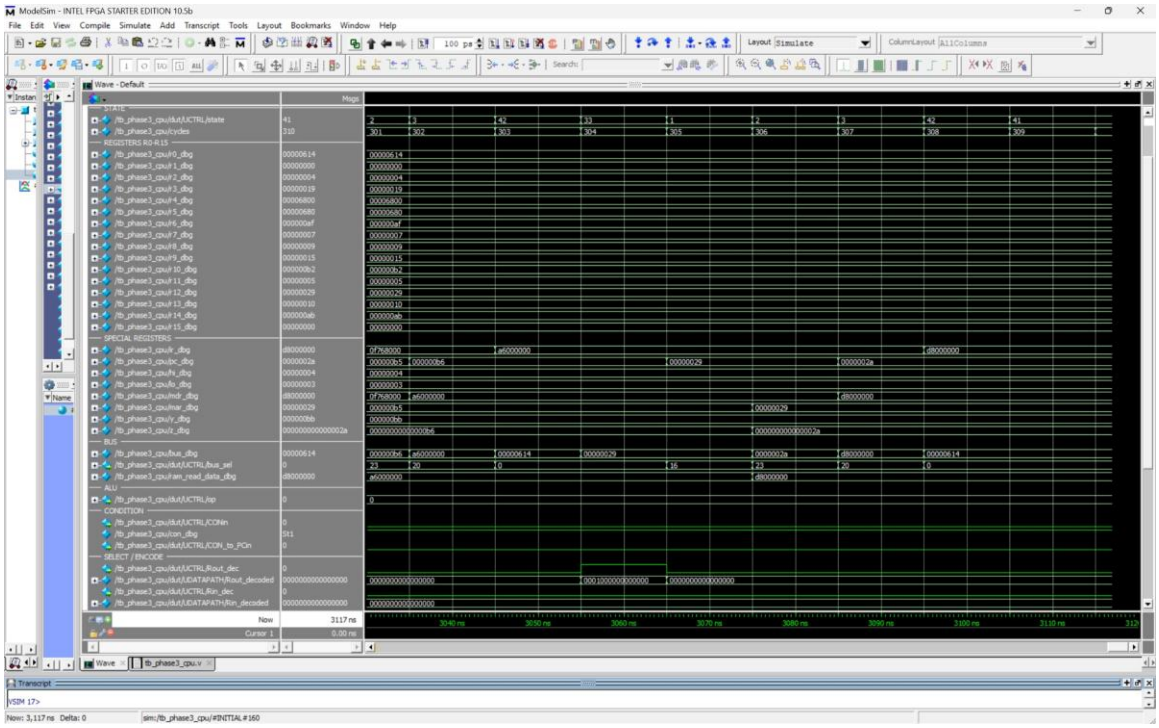
D.3 brmi not taken: IR=AA980003, CON=0, PC unchanged.



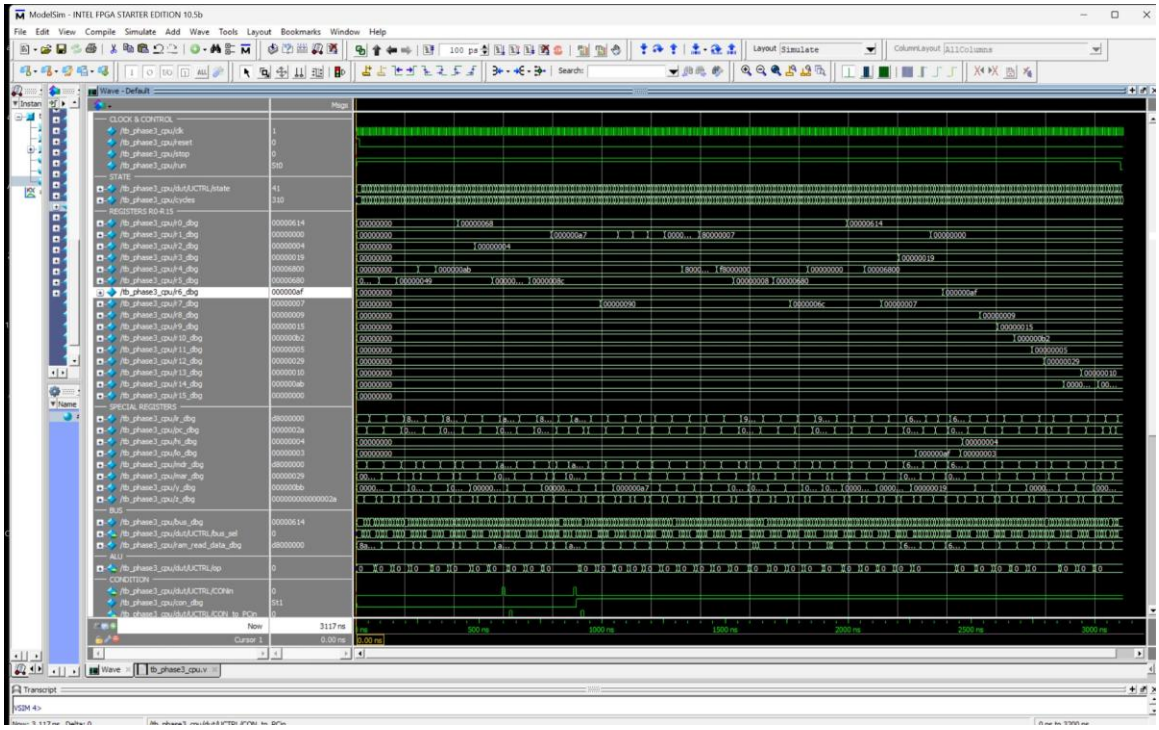
D.4 brpl taken: IR=A8900002, CON=1, PC jumps to 0x0E.



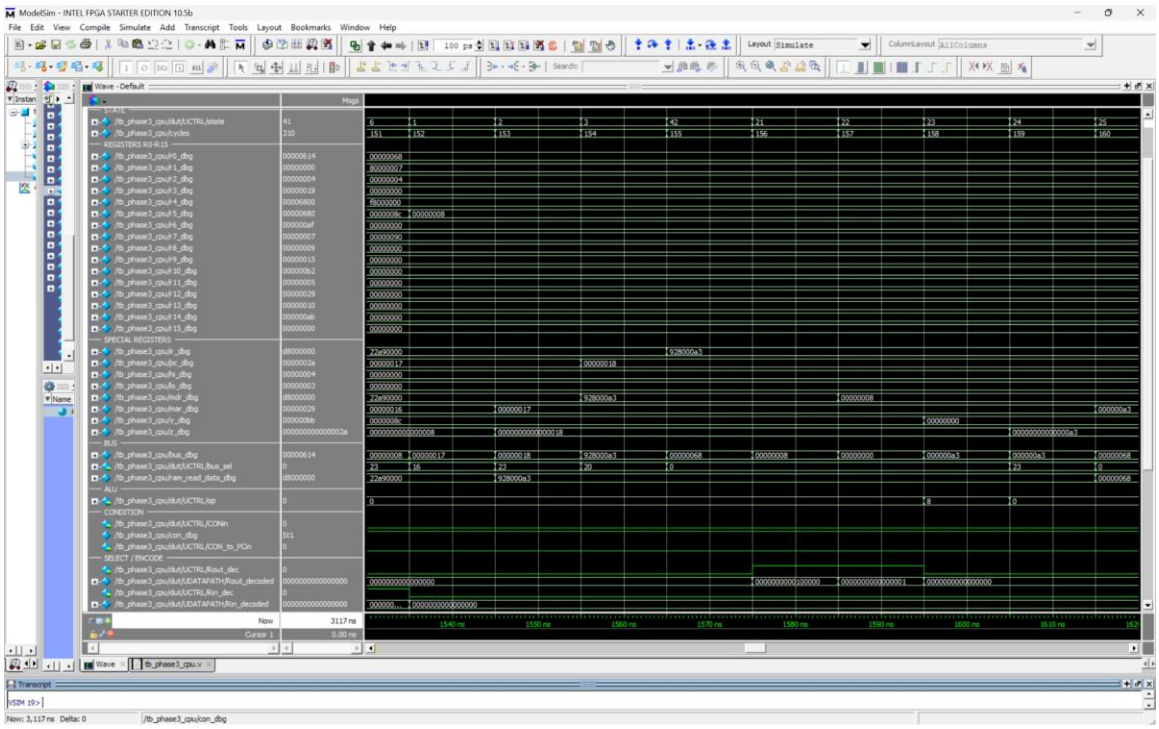
D.5 jal: R12 saves return address 0x29, PC jumps to subroutine at 0xB2.



D.6 jr: PC returns to 0x29, halt instruction executes.

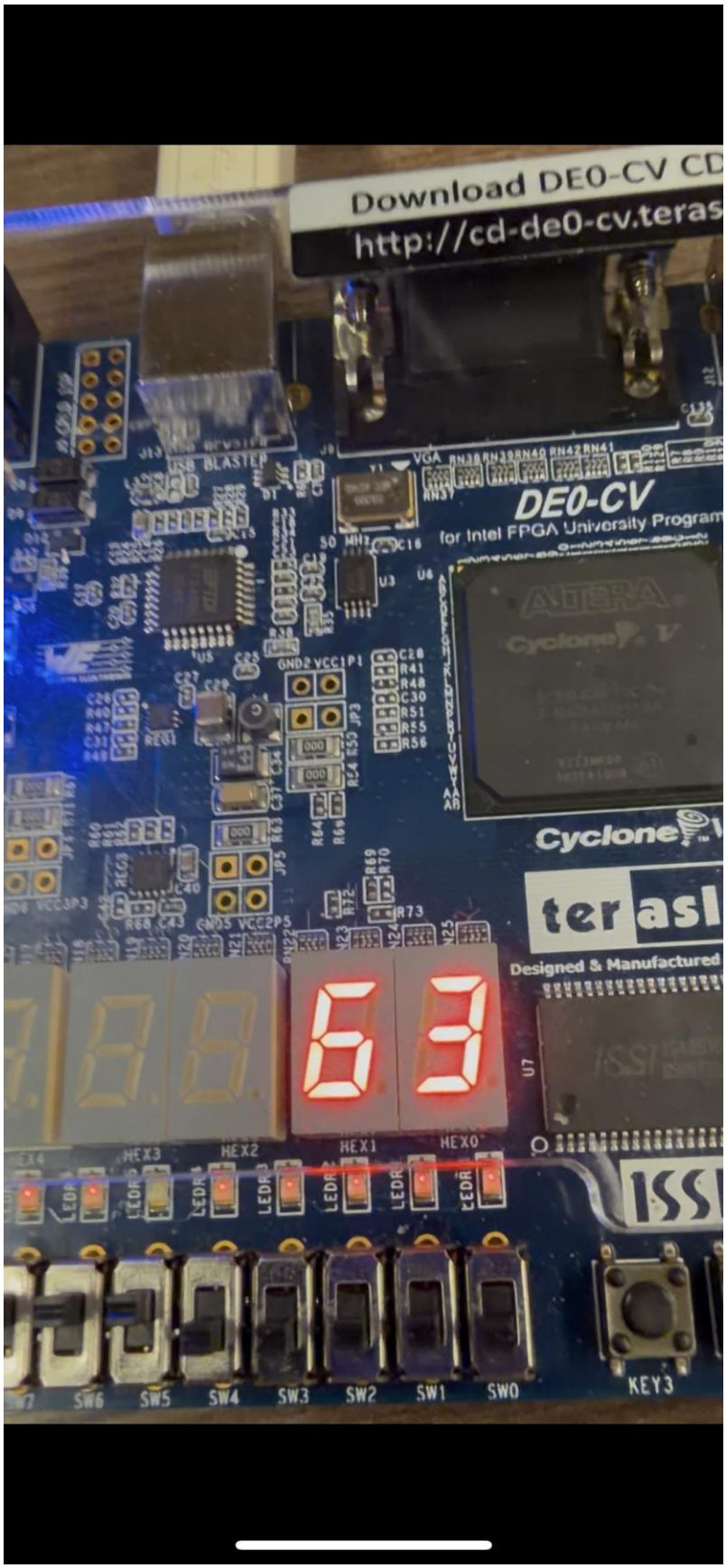


D.7 Full simulation waveform with final register values.

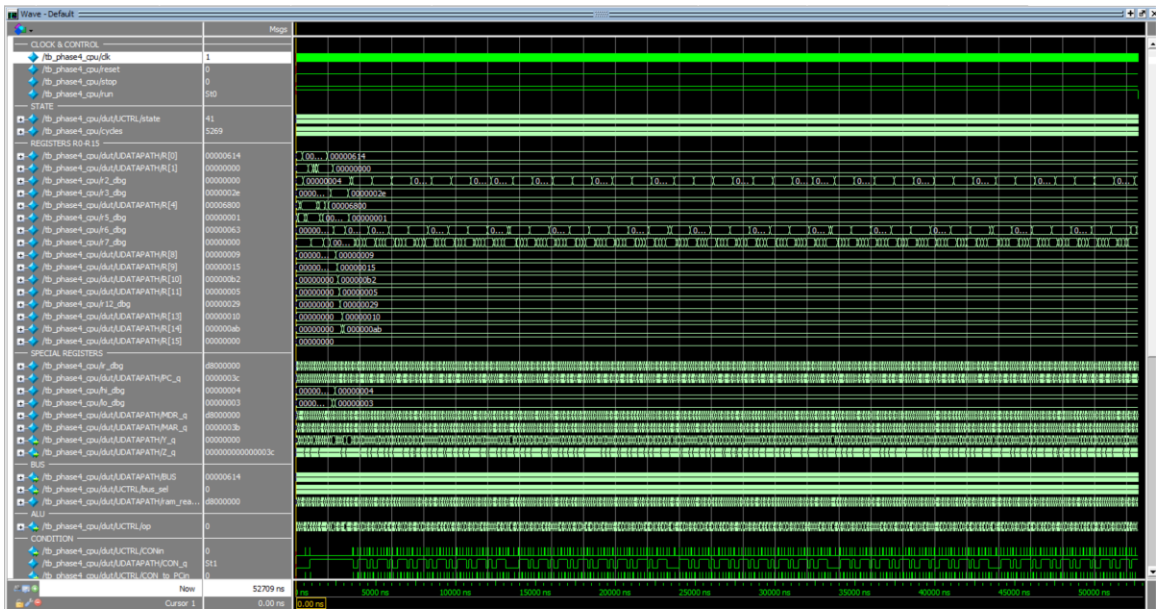


D.8 Store instruction: MAR and MDR values during write to memory.

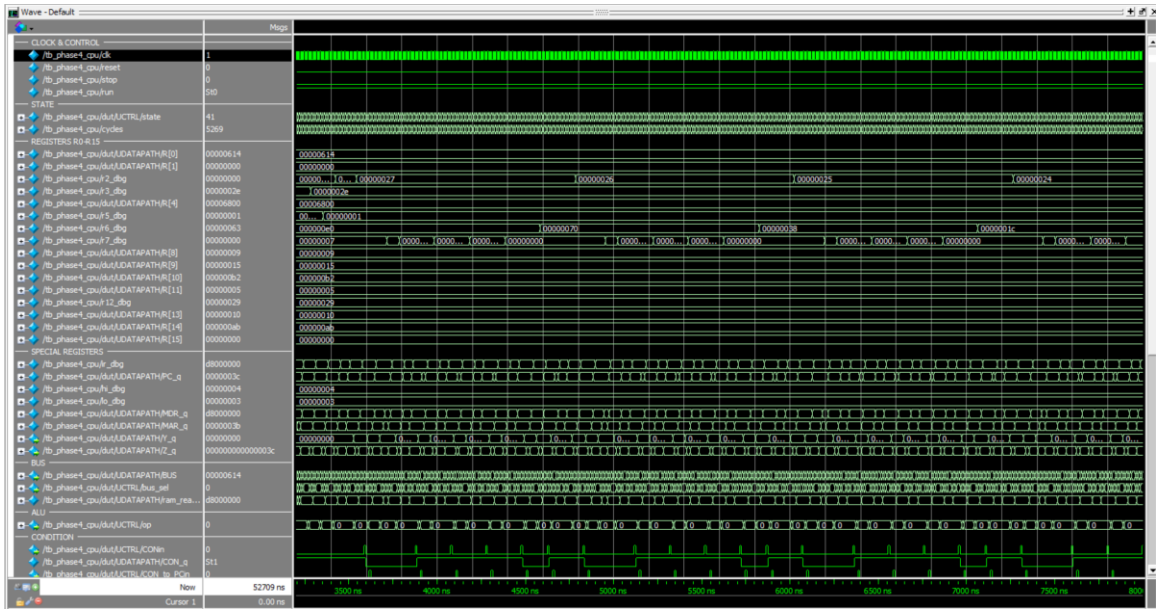
Appendix D.9: FPGA Board Photo


Download DE0-CV CD
http://cd-de0-cv.teras
DE0-CV
for Intel FPGA University Program
Cyclone V
terasic
Designed & Manufactured
HEX3
HEX2
HEX1
HEX0
SW7
SW6
SW5
SW4
SW3
SW2
SW1
SW0
KEY3

D.9 63 showing on hex with sw5-7 up and ledr5 off



D.10 Phase 4 full simulation waveform (0 to 52709 ns, 5269 cycles). All registers, output port, and control signals visible.



D.11 Phase 4 output display loop. R6 shifts right each iteration (E0, 70, 38, 1C). R2 loop counter decrements. CONin pulses show branch checks.